



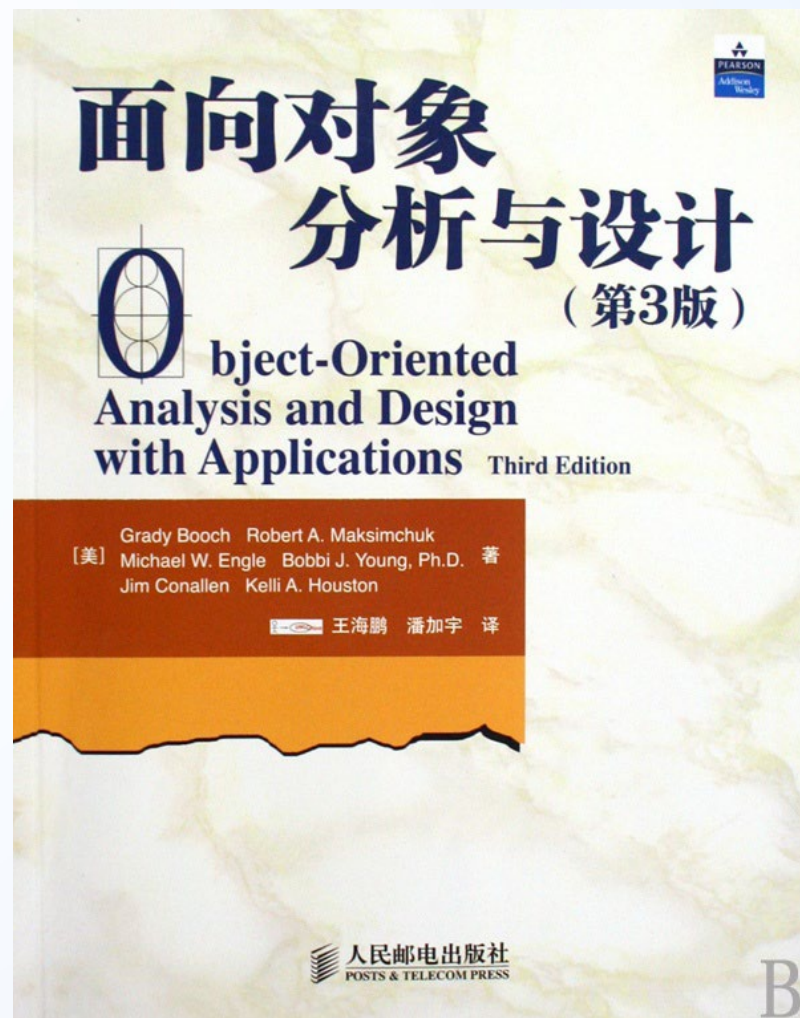
软件开发技术

第8章 基于UML的面向对象建模

浙江大学 机械工程学院 航空制造工程研究所

王青 李江雄

- 1. 统一建模语言 (UML)
- 2. 功能模型 (用例和用例图)
- 3. 静态模型
 - 3.1 类图
 - 3.2 对象图
- 4. 动态模型
 - 4.1 状态图
 - 4.2 顺序图
 - 4.3 协作图
 - 4.4 活动图
- 5. 其他





01

统一建模语言 (UML)

用面向对象方法成功开发软件的关键，是对问题域的理解。面向对象方法最基本的原则，是按照人们习惯的思维方式，用面向对象的观点建立问题域的模型，开发出尽可能自然地表现求解方法的软件。

用面向对象方法开发软件，通常需要建立3种形式的模型，它们分别是：

- **功能模型**：描述系统功能的模型；
- **静态模型**：描述系统静态特性、结构组织关系；
- **动态模型**：描述系统动态的、变化的特性。

这三种模型从三个不同的侧面全面反映对目标系统的需求。

功能模型表示了系统的功能需求，它指明了系统应该“**做什么**”，因而更直接反映了用户对目标系统的需求。如用例图。

静态模型表示静态的、结构化的系统的“**数据**”组织。如类图，对象图。

动态模型表示瞬时的、行为化的系统“**控制**”流程，它规定了静态模型中对象的合法变化序列。如状态图，顺序图，活动图，协作图等。

UML: Unified Modeling Language 是对软件系统中的**制品**进行可视化、详述、构造和文档化的语言。

制品 (artifact) 是指软件开发过程中产生的各种产物, 如模型、源代码、测试用例等。

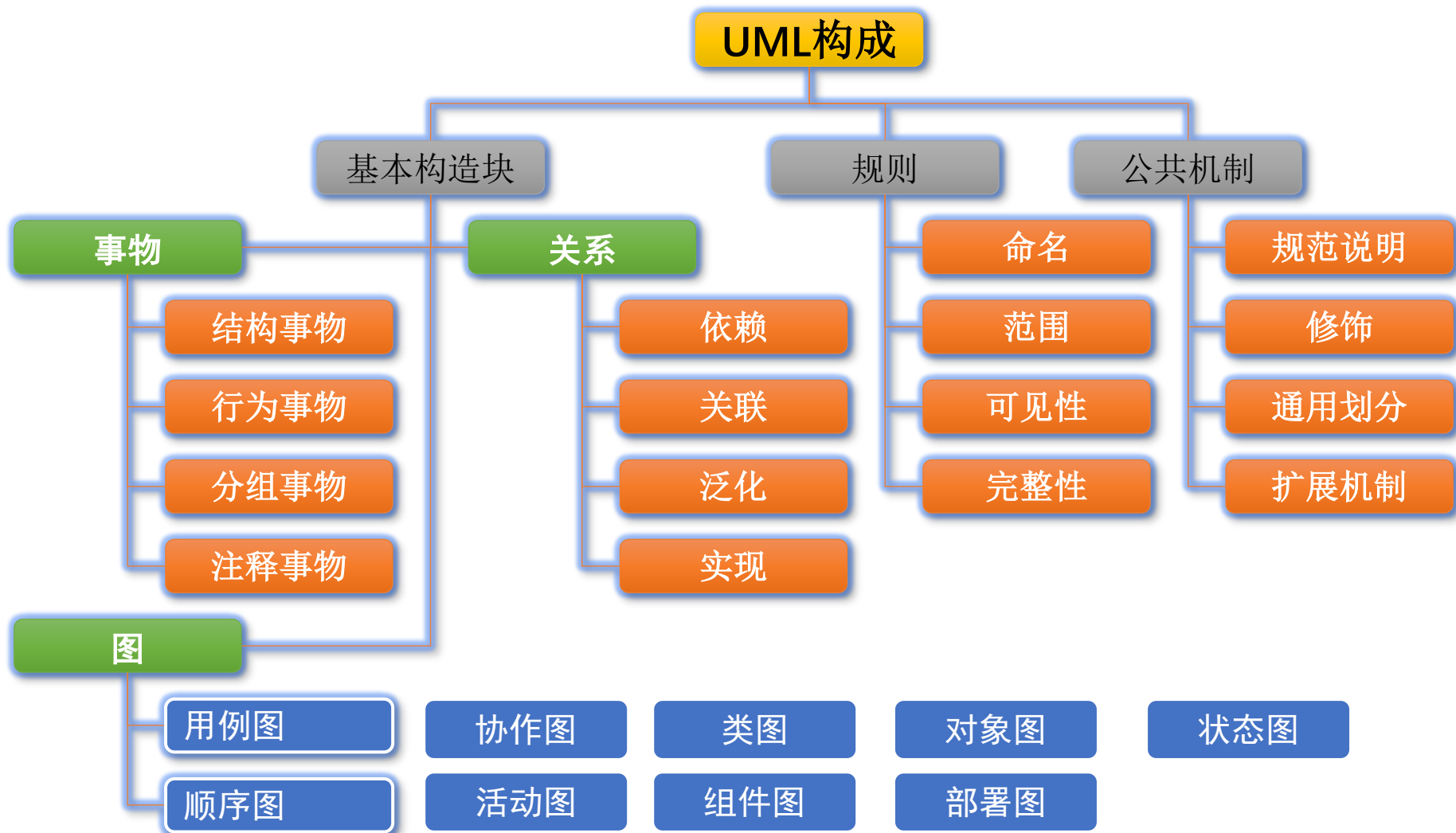
UML是由著名面向对象技术专家G. Booch / J. Rumbaugh / I. Jacobson发起, 在Booch方法、OMT (Object Modelling Technique) 方法和OOSE (Object-Oriented Software Engineering方法基础上,发展起来的。

UML绘制工具: **Enterprise Architect, Visual Paradigm, MS Visio**等

■ UML特点

- 统一的标准, UML已被OMG (Object Management Group) 接受为标准的建模语言;
- 面向对象, UML是支持面向对象软件开发的建模语言;
- 可视化, 表达能力强;
- 独立于过程, UML不依赖与特定的软件开发过程;
- 概念明确, 建模表示法简洁, 图形结构清晰, 容易掌握和使用。

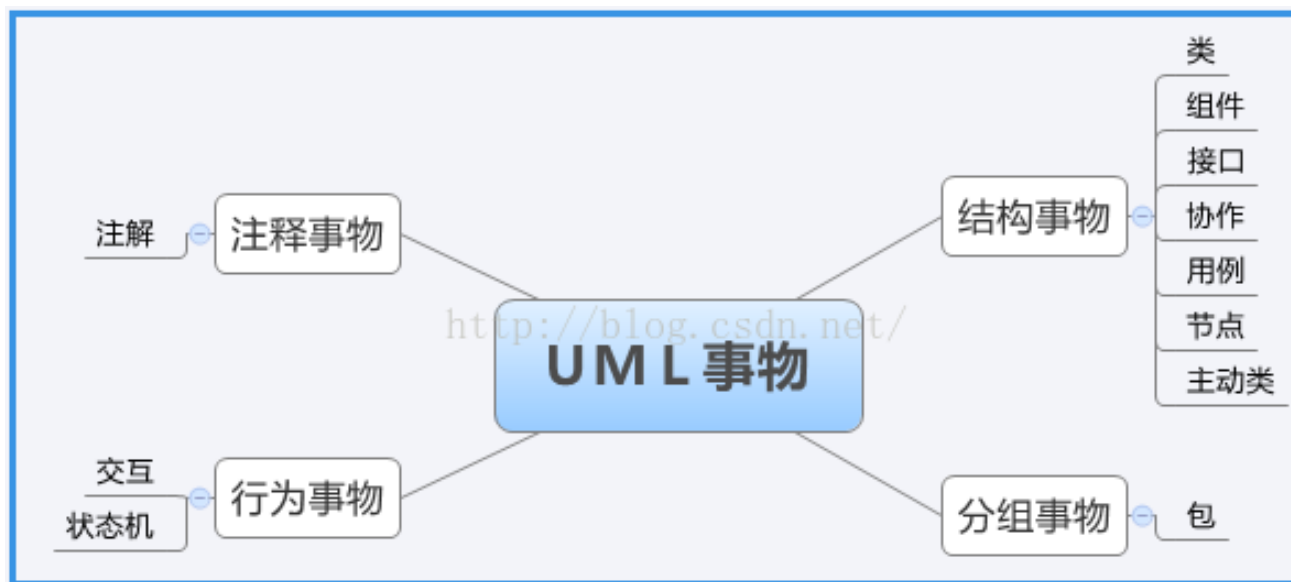
UML构成



UML的事物

UML有三个基本构造块，分别是事物，关系和图，UML事物有四种，分别是：

- ◆ 结构事物
- ◆ 行为事物
- ◆ 分组事物
- ◆ 注释事物



UML的事物

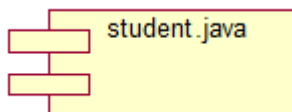
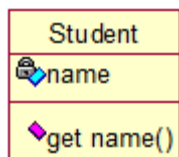
◆ 结构事物 (Structural thing)

主要描述概念的或物理元素等静态成分，包括类、主动类、接口、对象、用例、参与者、协作、构件和节点。

①类 (class)：类是具有相同属性/相同操作的一组对象的集合。用一个矩形来表示，通常矩形中写有类的名称/类的属性和类的操作。

②组件 (component)：组件是系统中物理的/可替代的部件，描述了一些逻辑元素（如类/接口）的物理包，由一个带有小方框的矩形表示。通常在矩形中只写该组件的名字。

③接口 (interface)：接口描述了一个类或组件的一个服务的操作集，或者说，接口描述了类或组件对外的/可见的动作。一个类可以实现一个或多个接口。接口用一个带有名称的圆表示。接口很少单独存在，而是依附于实现接口的类或组件。



UML的事物

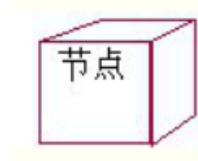
◆ 结构事物 (Structural thing)

④协作 (collaboration) :协作是一组类，接口和其他元素的群体，它们共同工作，提供比各组成部分的功能总和更强的合作行为。与组件不同，协作不能拥有自己的事物，而只能引用其他地方定义的类/接口/组件/节点等结构事物，即协作式系统体系结构中的概念组块而不是物理组块。协作用一个包含名称的虚线椭圆表示。

⑤用例(use case): 用例是对一组序列动作的描述，系统执行这些动作将对用例的参与者 (actor) 产生可观察的结果。用实线的椭圆表示，参与者用一个人形的图案表示。

⑥节点 (node) :节点是一个物理元素，它在运行时存在，代表一个可计算的资源，比如说一台数据库服务器。节点用一个立方体来表示。

⑦主动类 (active class) : 主动类能够启动控制活动，因为它的对象至少拥有一个进程或线程。主动类的表示也是使用一个矩形，只是最外面的边框使用粗线。

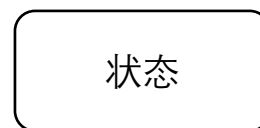


UML的事物

◆ 行为事物

结构事物描述的是模型的静态部分，而行为事物描述的是模型的动态部分。一共有两类主要的行为事物。

①交互 (interaction)：对象都不是孤立存在的，它们之间通过传递消息进行交互。在图形上，交互的消息通常用带箭头的直线表示。由在特定的上下文环境中共同完成一定任务的一组对象之间传递的消息组成，其涉及的元素包括消息、动作序列和链。



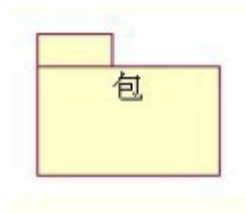
②状态机 (state machine)：一个状态机是一个行为，它说明对象在它的生命周期中响应时间所经历的状态序列以及它们对那些事件的响应。状态是指在对象的生命周期中满足某些条件/执行某些活动或等待某些事件时的一个条件或状况。一个事件的到来，能够触发一个状态的转换。

UML的事物

◆ 分组事物

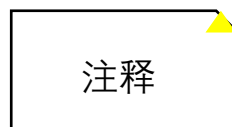
分组事物是UML模型中负责分组的部分，可以把它看作一个独立的盒子，每个盒子里面的对象关系相对复杂，而盒子与盒子之间的关系相对简单。最主要的分组事物是包。

包 (package) 是把元素组织成组的机制。结构事物/行为事物甚至其他的分组事物都可以放进包内。包纯粹是概念上的，不是物理上的，只在软件开发时存在，不存在于软件中。在图形上，包用一个在左上角带有一个小矩形的大矩形表示。

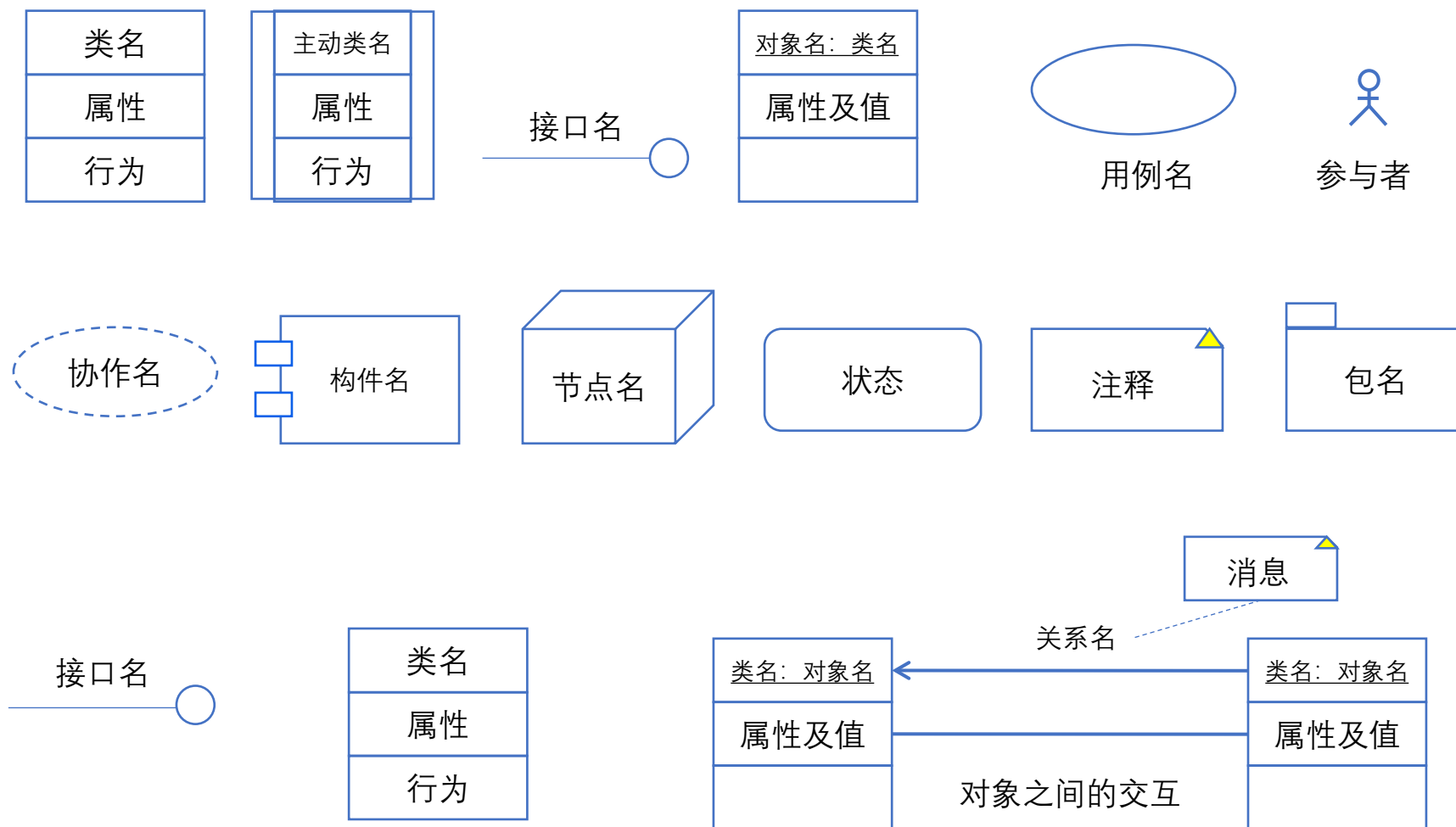


◆ 注释事物 (Annotational thing)

注释事物是UML模型的解释成分，它们用来描述和标注模型中的任何元素。通常可以用注释修饰带有约束或者解释图。



UML基本事物的图形表示



UML关系及其图形表示

UML 中类与类, 类与接口, 接口与接口之间的关系有:

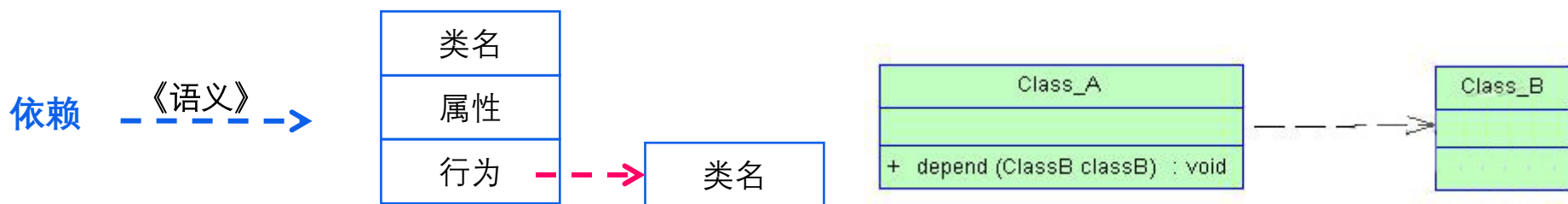
- ◆ 依赖(dependency)关系,
- ◆ 关联(association)关系(关联、 聚合、 包含),
- ◆ 泛化(generalization) 关系,
- ◆ 实现(realization)关系。

UML关系及其图形表示

◆ 依赖关系 (Dependency)

依赖关系总是单向的，用带箭头的虚线表示，如A -.-> B (A依赖于B)。它描述一个对象的修改会导致另一个对象的修改。可以理解为，就是一个类A使用到了另一个类B，而这种使用关系是具有偶然性的、临时性的、非常弱的，但是B类的变化会影响到A；比如某人要过河，需要借用一条船，此时人与船之间的关系就是依赖；

表现在代码层面，为类B作为参数被类A在某个method方法中使用。在java 中，依赖关系体现为：局部变量、方法中的参数和对静态方法的调用

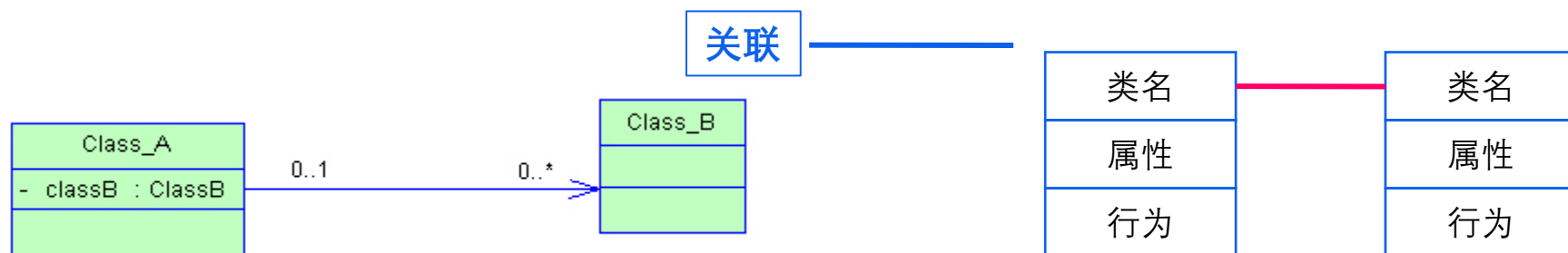


◆ 关联关系 (association)

关联关系是用一条直线表示的，如A——B。它描述不同类之间的结构关系，它在一段时间内将多个类的实例连接在一起。这与依赖关系是不同的，依赖关系通常表示两个实例之间的临时关联关系。可以使用关联关系表示一个对象了解其他对象，简单一点说，关联关系描述了某个对象在一段时间内一直“知道”另一个对象的存在。例如：A对象保存了B对象ID，因此A对象“知道”B对象的存在。

表示类与类之间的联接，它使一个类知道另一个类的属性和方法。

关联可以使用单箭头表示**单向关联**，使用双箭头或不使用箭头表示**双向关联**，不建议使用双向关联。

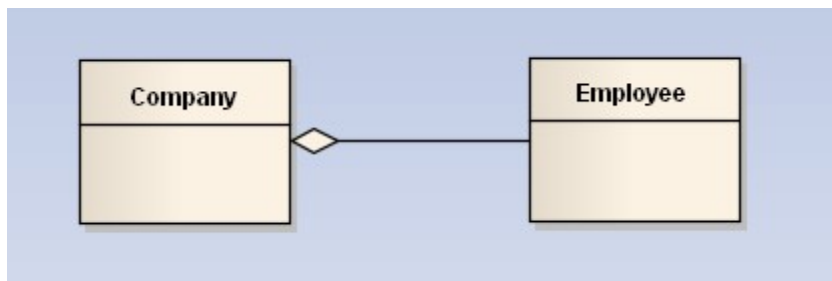


UML关系及其图形表示

◆ 聚合关系 (Aggregation)

关联关系可分为一般关联关系、聚合关系(Aggregation)和组合关系(Composition)。

聚合(Aggregation)：表示has-a的关系，是一种不稳定的包含关系。较强于一般关联，有整体与局部的关系,并且没有了整体,局部也可单独存在。如公司和员工的关系，公司包含员工，但如果公司倒闭，员工依然可以换公司。在类图使用空心的菱形表示，菱形从局部指向整体。

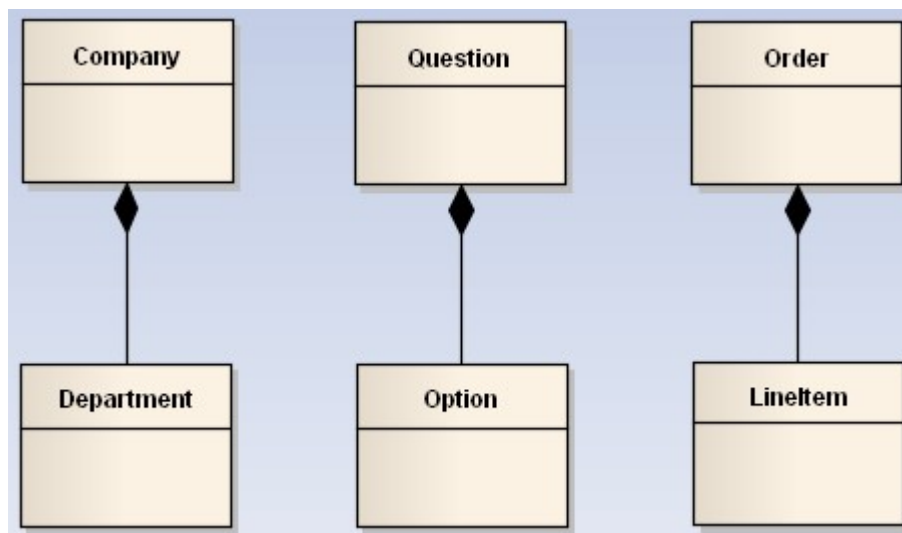


UML关系及其图形表示

◆ 组合关系 (Composition)

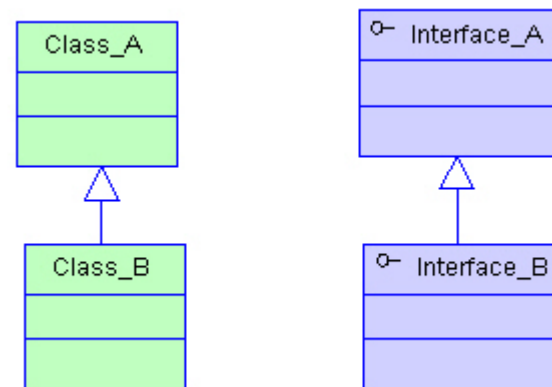
关联关系可分为一般关联关系、聚合关系(Aggregation)和组合关系(Composition)。

组合(Composition)：表示contains-a的关系，是一种强烈的包含关系。组合类负责被组合类的生命周期，是一种更强的聚合关系，部分不能脱离整体存在。如公司和部门的关系，没有了公司，部门也不能存在了；调查问卷中问题和选项的关系；订单和订单选项的关系。在类图使用实心的菱形表示，菱形从局部指向整体。



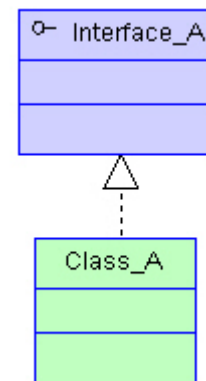
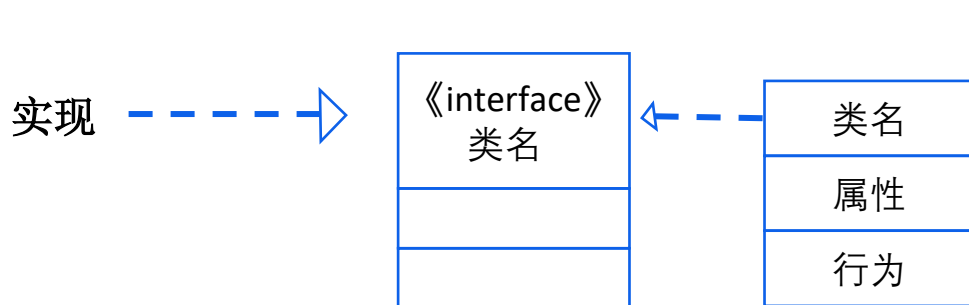
◆ 泛化 (generalization)

表示父类与子类之间的继承关系



◆ 实现 (implement)

是泛化关系和依赖关系的结合，一般表示接口和实现它们的类或构件之间的关系、用例和实现它们的协作之间的关系；在Java中，此类关系通过关键字implements明确标识，在设计时一般没有争议性。





02

功能模型

功能模型用于表达系统的“功能”需求，它指明了系统应该“做什么”，因此能直接反映用户对目标系统的需求。

在结构化软件开发中，用“数据流图”来表达目标系统的功能需求。在面向对象开发中，利用功能模型来表达系统的功能需求，能更有利于帮助软件开发人员和用户自身深入地理解问题域。此时，数据流图所能表达的信息，已经能够包含于后面所介绍的静态模型和状态模型中，

UML中，用用例图来表达系统的功能模型。

UML的用例图是进行需求分析和建立功能模型的强有力的工具。在UML中把用用例图建立起来的系统模型称为**用例模型**。

用例模型通过一系列用例描述系统的不同用户的使用情况。能克服不同类型用户只知道自己应该如何使用系统而没有人准确地知道系统的整体运行情况的缺陷。因而能全面地获取用户需求，它能有效地回答“系统应该为每类用户作什么”的问题。

◆用例的概念 USE CASE

■用例定义

定义1：用例是对一个**活动者**（actor）使用**系统**的一项功能时所进行的**交互过程**的一个文字描述序列

定义2：用例是**系统**、子系统或类和外部**参与者**（actor）**交互的动作序列**的说明，包括可选的动作序列和会出现异常的动作序列。

■用例的作用

用例是对软件产品本身和软件产品的使用者（参与者、活动者、actor）之间的交互进行建模。用例代表系统中各个项目相关人员就系统的行为所达成的契约。

参与者（actor）是担负特定任务的用戶。是指系统以外的、需要使用系统或与系统交互的东西，包括人、设备、外部系统等。

用例描述的是系统一段连贯的功能

- 用例不是描述参与者与系统间的一个交互，而是一连贯的交互，这个交互过程应该能反映了一个完整的应用场景。
- 也就是说，用例涉及系统与其参与者之间的消息序列，而该消息序列在真实事件是有意义的。
- 用例把与一段程序功能相关的所有行为组织在一起
- 用例就是从用户的角度出发对如何使用系统（如何按照给定的消息序列进行交互）进行描述。

■ 用例 (use case) 的重要性

用例是一个能够让系统的用户在系统开发完成之前就能以他们自己的观点来看待系统的工具。

通俗地说,如果你要向你的用户描述你的系统将能为他做什么,他该怎么使用你开发的系统时,用用例模型将是一个比较好的选择。

同时,在开发组内部,用例模型是用户界面开发者设计人机交互的界面的主要依据。

■ 用例的表示

在UML中,用例用一个椭圆表示。

用例名在椭圆下方。一般用动宾结构或主谓结构



名称

■ 系统

在用例模型中,系统往往被看做一个提供用例的黑盒子,用例模型不描述系统如何工作,只描述它做什么工作以及系统的边界,边界划定系统的功能范围。

■ 参与者

参与者是指与系统交互的人或者其他系统,它代表外部实体。使用用例并且与系统交互的任何人或物都是参与者。

参与者代表一类角色,而不是某一个具体的人或物。事实上一个具体的人或物可能充当不同的角色,那么,在用例模型中,它属于不同的参与者。

一般我们可以将不同参与者理解为系统的不同用户。

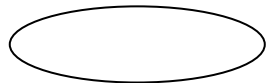
■ 用例举例

如对于一个学生学籍管理系统，可能有下列的使用功能需求

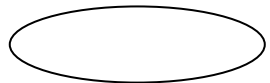
- 往系统中增加一个新到的学生
- 修改一个已在学学生的记录
- 删除一个已转学学生的记录
- 学生查询学生的记录



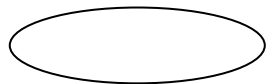
增加学生记录



修改学生记录



删除学生记录



查询学生记录

1. 插入储蓄卡
 2. ATM显示欢迎、请求输入数目信息
 3. 输入取款数目
 4. ATM吐出现金
 5. ATM吐出卡
 6. 取回卡
- 如果存款数不够、怎么处理
- 如果数目超限怎么处理
- 如果内部现金不够怎么处理

对于ATM机

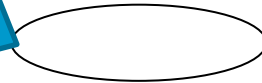
一个ATM机可至少四种使用情况：

- 往ATM机中补充现金
- 从ATM机中取现金
- 利用ATM机存款
- 对ATM机进行维修

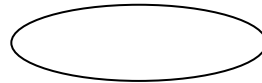
相应地可以用四个用例来表示



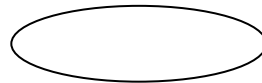
补充现金



取现金



存款

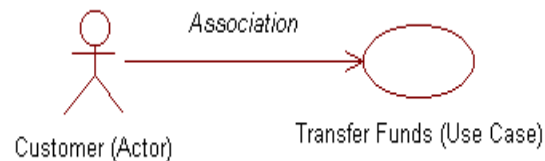
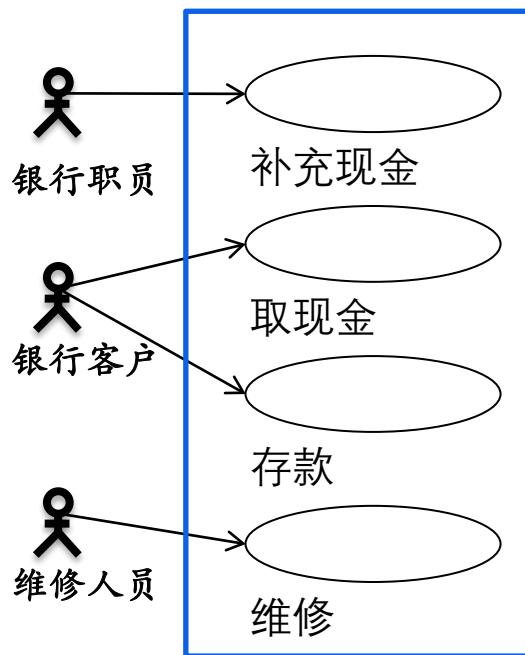
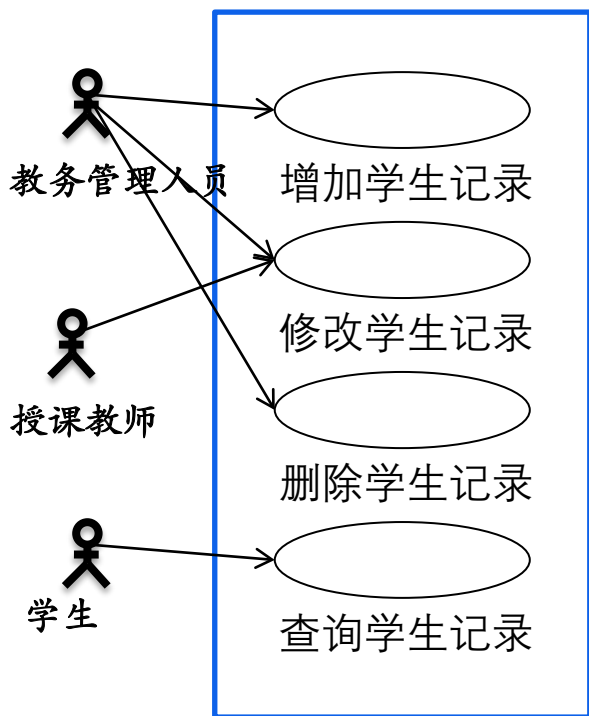


维修

通过进一步分析可能可以决定维修不是我们要考虑的范围。

◆用例图

用例图就是一组用例、参与者以及它们关系的图；
在UML中，一个用例模型由若干个用例图表示。



参与者与用例的关系表示法

- 用例用椭圆+用例名表示
- 参与者用棍形人表示
- 参与者与用例之间的联系用带名称的箭头表示。

■用例与用例之间存在关系

- 泛化：子用例继承父用例的行为和含义，子用例也可以增加新的行为或覆盖父用例的行为。
- 包含：其中一个用例包含了另一个用例的所有行为。
- 扩展：子用例继承父用例的行为和含义，但父用例规定了扩展点，子用例只能在扩展点处进行扩展。

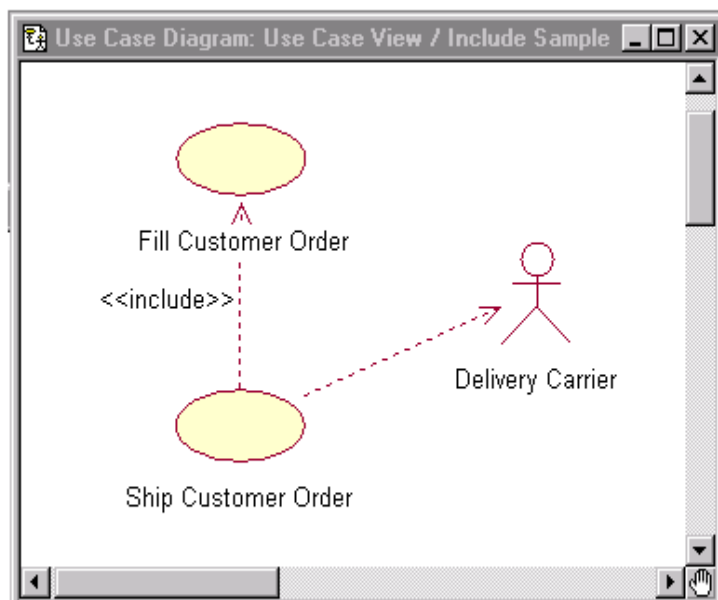
■参与者参与者之间也存在关系

- 泛化：参与者之间的泛化关系表示一般参与者与特殊参与者之间的关系。

包含关系:

Include Relationship Sample

The include relationship sample demonstrates how you can use an include relationship to connect use cases. The following sample illustrates the most important aspect of include behavior: an include relationship adds additional functionality not specified in the base use case.



做Ship Customer Order必须要
做Fill Customer Order

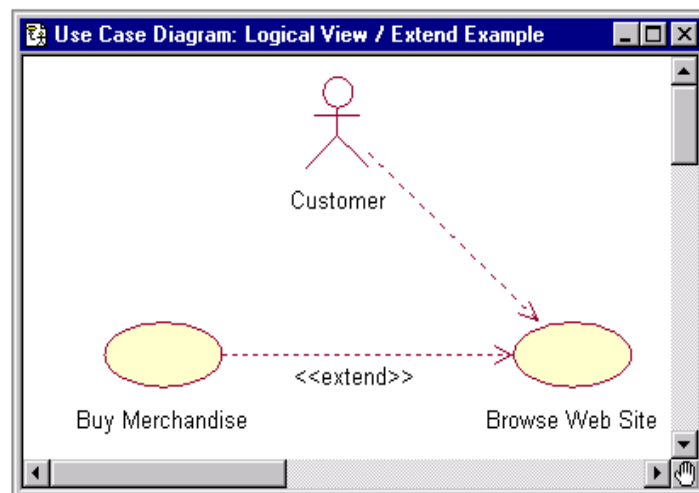
Notice the include relationship from **Fill Customer Order** to **Ship Customer Order**. The include relationship is significant because it pulls the functionality from the **Fill Customer Order** use case into the **Ship Customer Order** use case. Without the help of the **Fill Customer Order**, the order does not get shipped.

扩展关系:

The extend relationship sample demonstrates how you can use an extend relationship to connect use cases. The sample illustrates two important aspects of extend relationships:

- An extend relationship shows optional functionality or system behavior.
- A base use case does not need to acknowledge any specific extended use cases

The sample below shows a diagram containing an actor interacting with a web site. The **Customer** has the option of buying merchandise online as shown through the extend relationship.



做Browse Web Site可以
Buy Merchandise, 也可以不
Buy Merchandise

Notice the extend relationship between **Buy Merchandise** and **Browse Web Site**. The extend relationship is significant because it shows optional functionality. If the **Customer** desires, he/she can buy merchandise through the web site. However, the **Customer** may only browse through the web site without buying anything.

■ 用例的描述

在用例图中，一个用例用一个命名的椭圆表示，但如果没有对这个用例的具体说明，那么还是不清楚该用例到底会完成什么功能。

一般，用例采用自然语言描述参与者与系统交互时双方的行为，不追究形式化语言表达。因为用例最终是给开发人员、用户、项目经理、测试人员等不同类型的人员看的，如果采用形式化的描述，对大部分人来说会很难理解。

描述内容包括：

- 用例的目的
- 用例是怎样启动的
- 参与者和用例之间的消息是如何传递的
- 用例中除了主路径外，其他路径是什么
- 用例结束后的系统状态
- 其他需要描述的内容

用例描述的格式

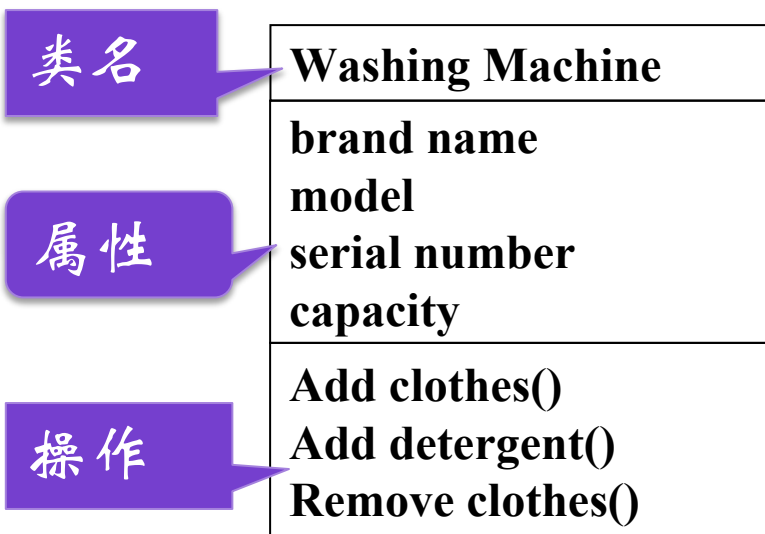
描述项	说明
用例名称	表明用户的意图或用例的用途，如“划拨资金”
标识符【可选】	唯一标识符，在文档别处可以用该标识符引用这个用例
用例描述	概述用例的几句话
参与者	与此用例相关的参与者列表
优先级	一个有序的排列，1 表示优先级最高
状态【可选】	用例的状态，通常为以下的几种之一：进行中，等待审查，通过审查或未通过审查
前置条件	一个条件列表，如果其中包含条件，则这些条件必须在访问用例之前得到满足
后置条件	一个条件列表，如果其中包含条件，则这些条件在用例完成后得到满足
基本操作流程	描述用例中各项工作都正常进行时的工作方式
可选操作流程	描述变更工作方式，出现异常或发生错误的情况下所遵循的路径
被泛化的用例	此用例所泛化的用例列表
被包含的用例	此用例所包含的用例列表
被扩展的用例	此用例所扩展的用例列表
修改历史记录【可选】	关于用例的修改时间，修改原因和修改人的信息
问题【可选】	与此用例的开发相关的问题列表
决策【可选】	关键决策的列表，将这些决策记录下来以便维护时使用
频率【可选】	参与者访问该用例的频率，如用户每日访问一次还是每月访问一次

(1) 类图

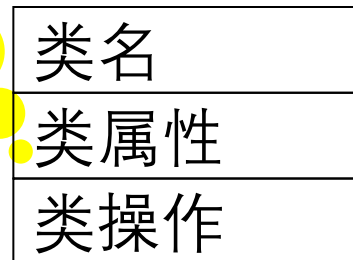
类 (Class) 是一类对象，拥有共同的结构、共同的行为和共同的语义。

例如：属于洗衣机类的事物都具有诸如：品牌、型号、序列号、容量等属性，这类事物的行为包括“加衣物”、加洗涤剂、开机、取衣物等操作。

在UML中用下列图来表示该类：

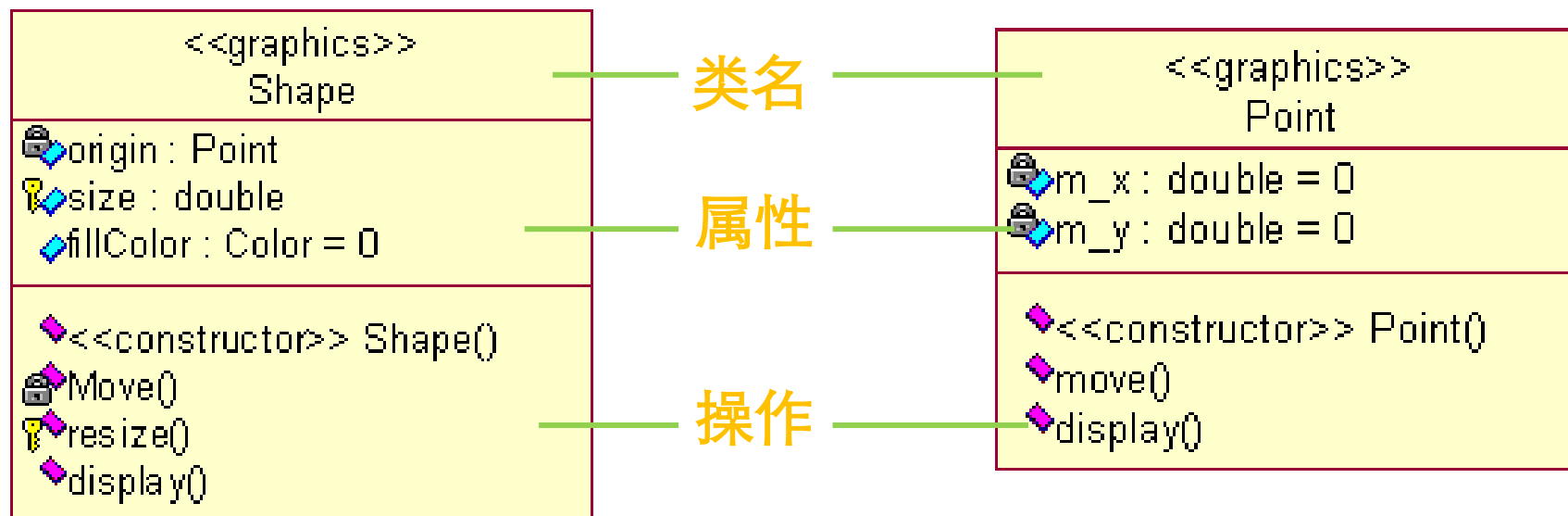


一个矩形方框代表类的图标，它被分成3个区域。最上面的是类的名称，中间为类的属性，下面的区域为类的操作。



类图就是这些表示类的图框及表示类之间关联的图框间连线所组成。

以下为Rational Rose中类图的UML表示：



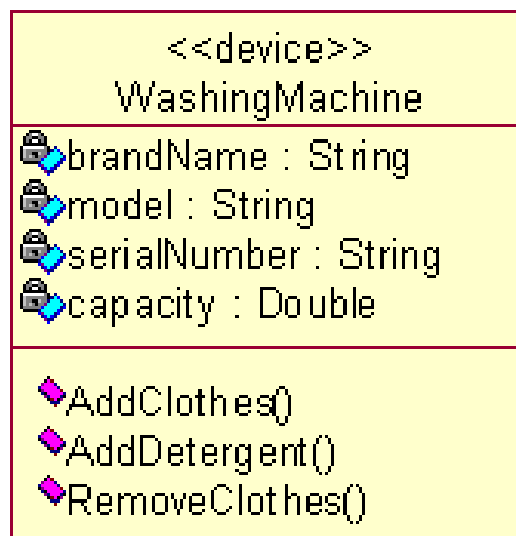
◆类名：

在定义类的时候，类的命名应尽量用应用领域的术语，应明确、无歧义，以利用开发人员和用户之间的相互理解和交流。

类的名称一般为名词。

类名的首字母一般用大写，若为多个名词组成，则一般每个单词的第一个字母为大写。

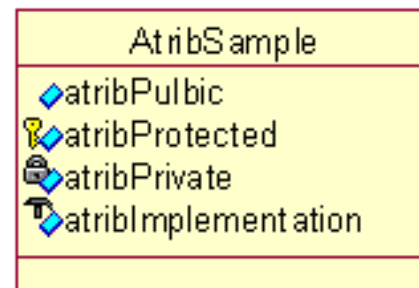
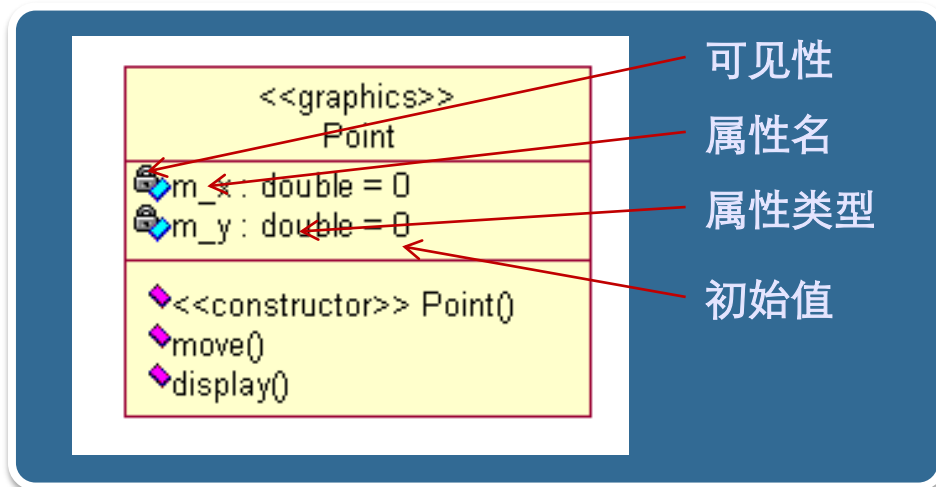
如：



◆ 类的属性

类的属性的格式为：

[可见性]属性名[:类型][初始值][{特性}]



可见性

➤ Public

The element is visible outside of the enclosing package and you can import it to other portions of your model. Operations are accessible to all clients.

➤ Protected

The element is accessible only to derived classes, friends, or to the class itself.

➤ Private

The element is accessible only to friends, or to the class itself.

➤ Implementation

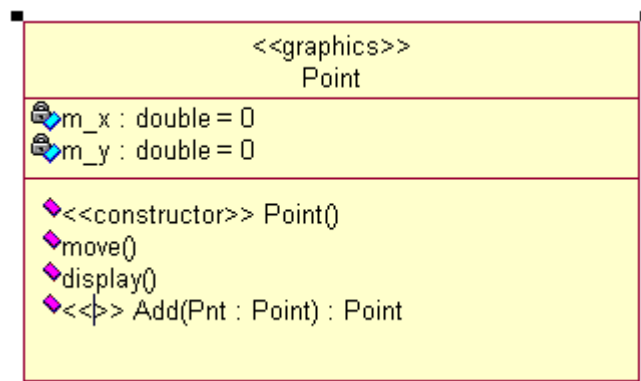
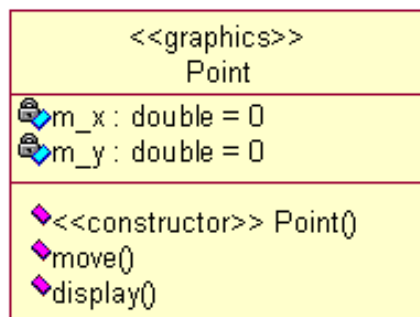
The element is visible only in the package in which it is defined. An operation is part of the implementation of the class.

◆类的操作

操作用于修改、检索类的属性或执行某些动作，操作通常也称为功能。但是它们被约束在类的内部，只能作用到该类的对象上。

其格式为：

[可见性]操作名[(参数列表)][[:返回类型]][{特性}]



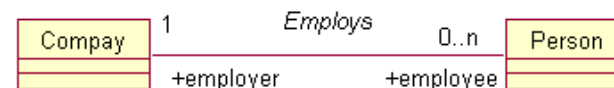
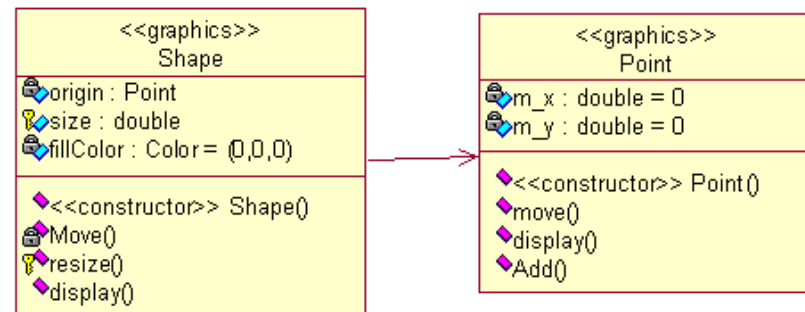
带参数的操作

◆ 类与类之间的关系

关联 Association Relationship

An association provides a pathway for communication. The communication can be between use cases, actors, classes or interfaces. Associations are the most general of all relationships and consequentially the *most semantically weak*. If two objects are usually considered independently, the relationship is an association.

By default, the association tool on the toolbox is uni-directional and drawn on a diagram with a single arrow at one end of the association. The end with the arrow indicates who or what is receiving the communication.



处理shape时一般要用point信息，所以需要向point发消息，调用其某些操作，但是处理点时，并不需要shape的信息，所以关联是单向的。

■ 关于关联的一些概念

➤ 关联名

如果关联是双向的可以取一个名称，一般用来描述关联的作用。（可以不取名称）

➤ 角色名

描述关联两端类以何种身份参与关联

➤ 多重性

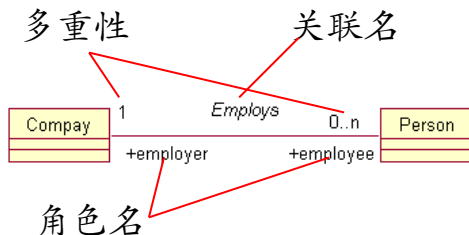
表示可以有多少个对象参与关联

➤ 关联类

为了说明关联的性质，可能需要增加一些附加信息，通过引入一个关联类来记录这些信息，关联中的每个链接与关联类的对象相对应。

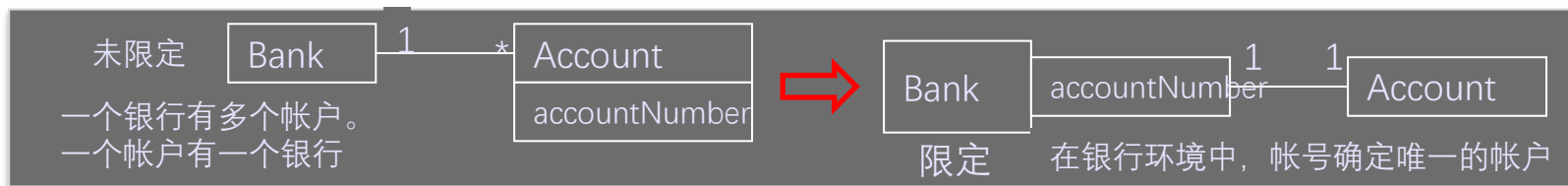
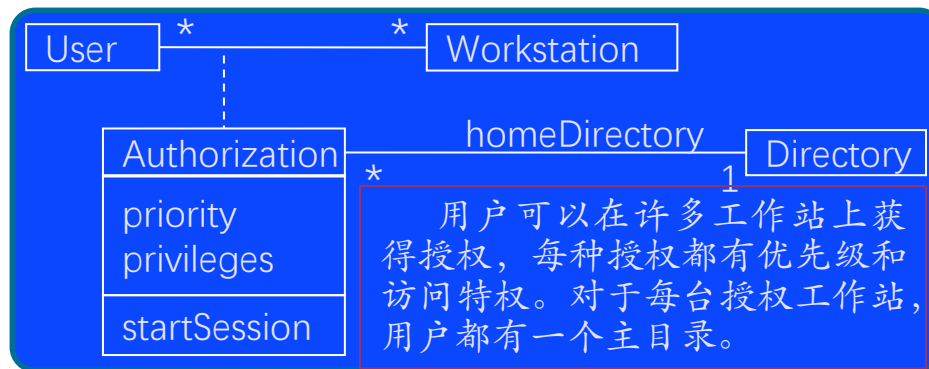
➤ 限定关联

限定关联是这样一种关联，其中被称为限定符的属性会消除在关联“多”端上的对象歧义。限定符在目标对象之间进行选择，将有效的多重性从多降为1。目标多重性为1或0..1的限定关联确定了一条准确的路径，可以从源对象找到目标对象。



图中多重性表示：

- 一个雇员只能为一家公司所雇佣；
- 一家公司可以雇佣多个雇员。



■ 泛化

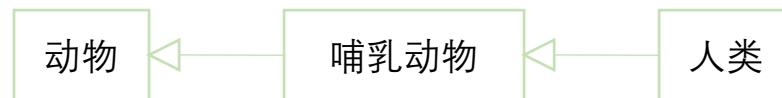
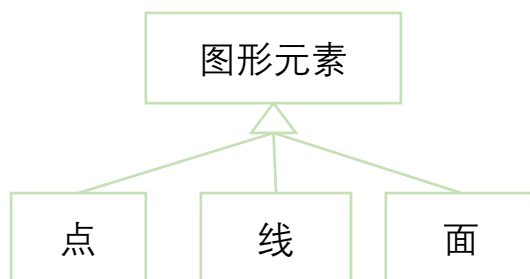
泛化指父类与一个或多个子类之间的继承关系。特殊与一般的关系。

泛化利用类的相似性和差异性来组织类。同样描述对象的结构，父类拥有公共属性、操作和关联，子类增加了特定的属性、操作和关联。

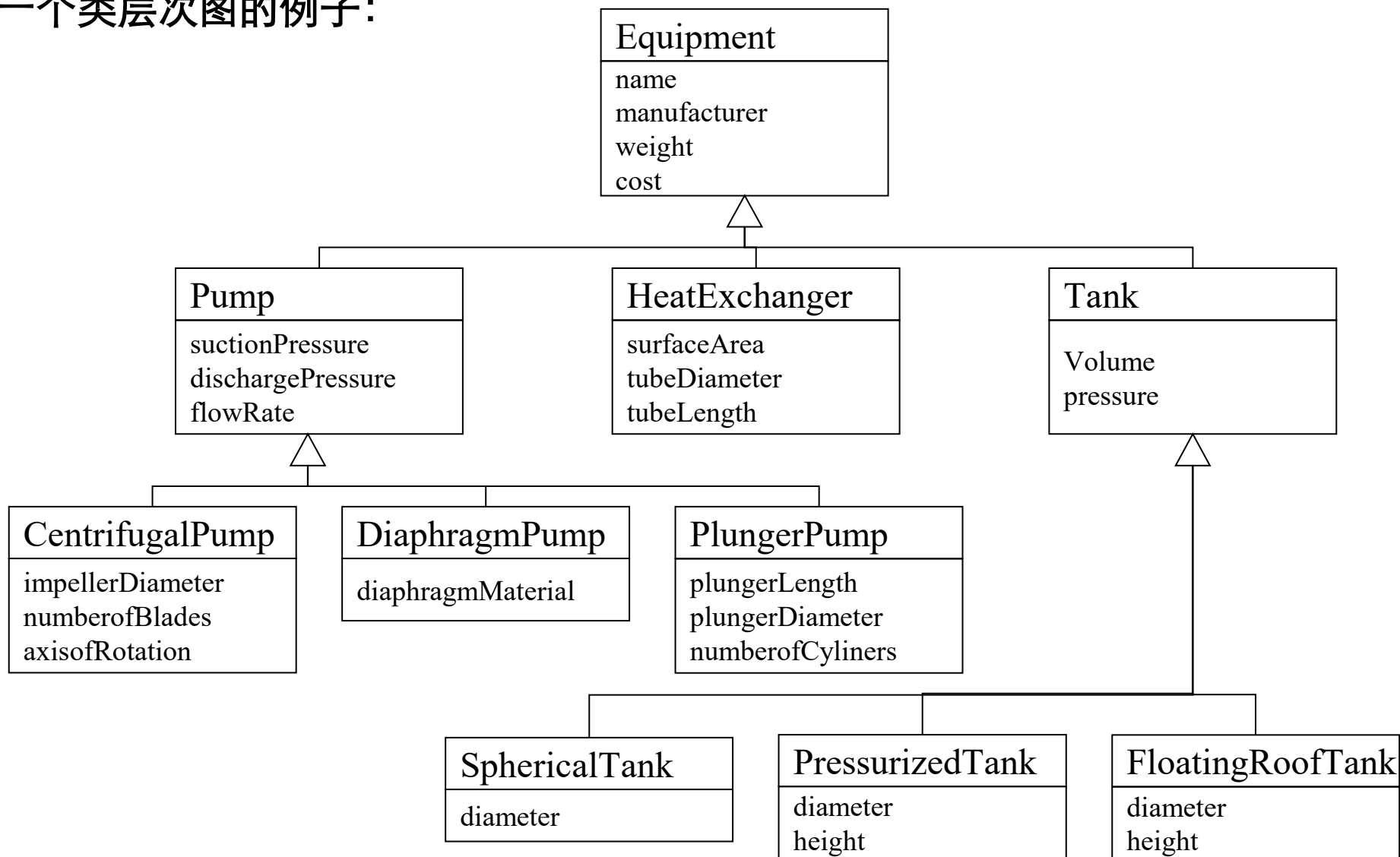
每个子类继承了其父类的特征。

泛化有时被称为” is a kind of” 或 “is a” 关系

泛化可以将类分级，每个子类有一个直接父类。泛化可以有 multiple 层。

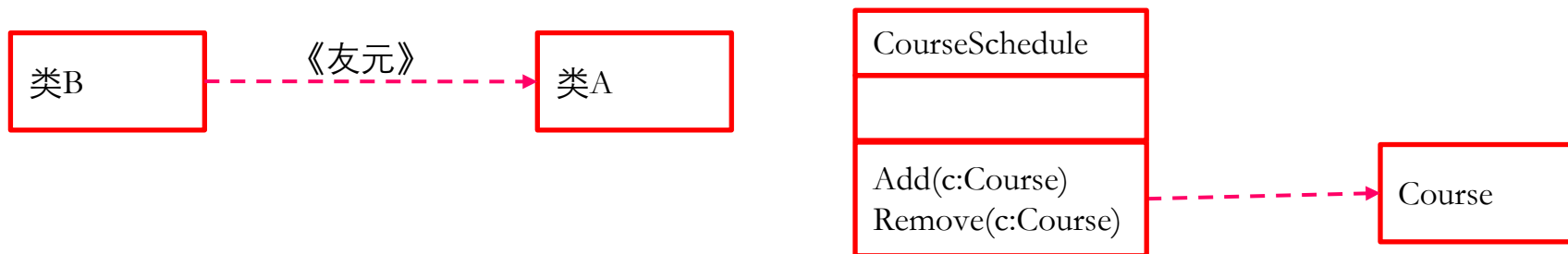


一个类层次图的例子：



■ 依赖

- 依赖关系描述两个模型元素（类、用例等）之间的语义连接关系：其中一个模型元素是独立的，另一个模型元素依赖于独立的模型元素，如果独立的模型元素改变了，将影响依赖于它的模型元素。例如，一个类使用另一个类的对象作为操作参数，一个类用另一个类的对象作为它的数据成员，一个类向另一个类发送消息等，这样的两个类之间都存在依赖关系。
- 在UML的类图中，用带箭头的虚线连接有依赖关系的两个类，箭头指向独立的类，在虚线上可以带一个版类标签具体说明依赖的种类。
- 例：A对象保存了B对象的ID，但A对象对B对象没有操作，这时A仅仅是“知道”B对象，应当用关联关系；如果A对象使用了B对象的属性或方法，则B的修改会导致A的修改，这时A依赖于B。



■ 实现关系

- 实现是泛化关系和依赖关系的结合，也是类之间的语义关系，通常在以下两种情况出现实现关系
- (1)接口和实现它们的类或构件之间； (2) 用例和实现他们的协作之间。

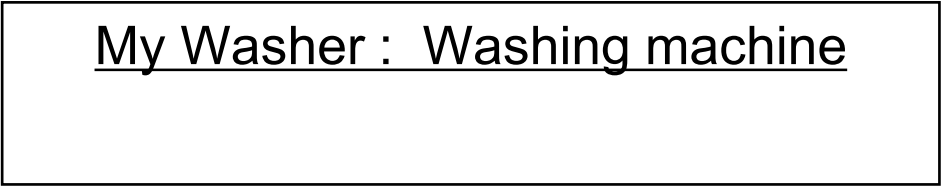
(2) 对象图

对象（object）是一个类的实例，是具有具体属性值和行为的一个具体的事物。

如，某台洗衣机的具体值为：

品牌：小天鹅，型号：BD4010， 序列号：GL080414001， 容量：6公斤

在UML中，也用一个矩形来表示对象，与类不同的是，对象的名称写法为：对象名：类名， 如：



My Washer : Washing machine

The diagram shows a rectangular box representing an object. Inside the box, the text 'My Washer : Washing machine' is written. The text is underlined, indicating it is an object name followed by its class name.

为什么要进行动态分析

- ※ 用例模型中对系统的功能(用例)进行了描述, 但所关注的重点是: 系统能做什么 (*What*)
- ※ 问题: 怎么做 (*How*) 才能实现系统的每一种功能?
- ※ 在静态模型中, 确定了所有构成系统的类, 并且确定了类之间的关系以及类的属性
- ※ 问题: 类之间的关系是否正确? 类的操作该如何确定?

动态分析的任务：

- ※ 通常用来描述用例的**行为实现**
- ※ 显示用例实现/类操作（方法）涉及的对象和对象间的**消息传递情况**
- ※ 用于对系统中一组对象群体的行为建模
- ※ 主要用来表达对象之间的**动作协作关系**

动态分析模型描述系统的动态行为，显示对象在系统运行期间不同时刻的动态交互。动态分析模型包括下列图形：

❖ 状态图

❖ 活动图

行为图

❖ 序列图

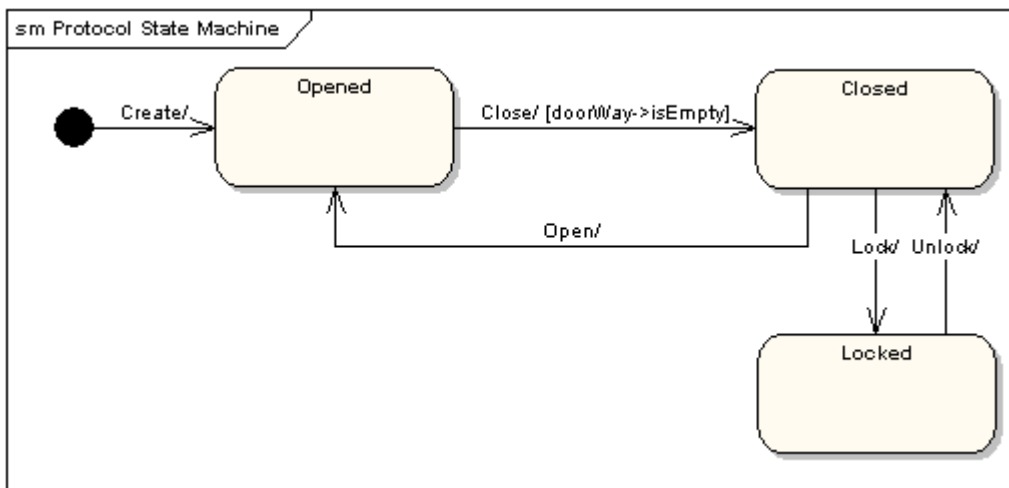
❖ 协作图

交互图

◆4.1 状态图

状态图（Statechart Diagram）主要用于描述一个对象在其生存期间的动态行为，表现为一个对象所经历的状态序列，引起状态转移的事件（Event），以及因状态转移而伴随的动作（Action）。一般可以用状态机对一个对象的生命周期建模，状态图用于显示状态机（State Machine Diagram），重点在与描述状态图的控制流。

如下图例子，状态机描述了门对象的生存期间的状态序列，引起转移的事件，以及因状态转移而伴随的动作（Action）。



状态有：Opened、Closed、Locked
事件有：Open、Close、Lock和Unlock
注意：

1、并不是所有的事件都会引起状态的转移，比如当门是处于【Opened】状态，不能进行【Lock】事件。

2、转移（Transition）有监视条件（guard condition），比如只有doorWay->isEmpty 条件满足时，才会响应事件。

■对象的状态

状态是对对象取值和链接的抽象。根据对象的总体行为，将取值和链接的集合组成一个状态。

状态反映的是“对象取值和链接”，实际上是对对象的某些属性的不同取值对应对象的不同状态

例如根据某公司的资产是否超出了其债务，公司的状态要么是有偿还能力，要么是无偿还能力。

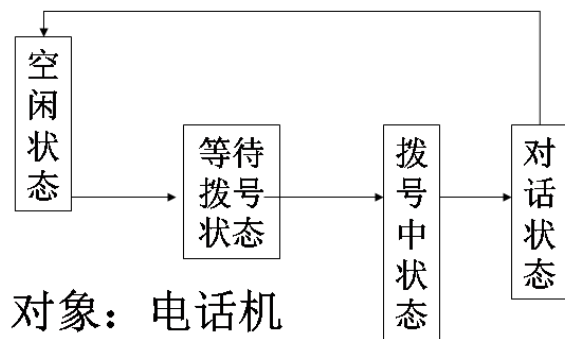
① 通过响应事件对象的状态会发生改变。

对于打电话，在拿起话筒的瞬间，电话进入等待拨号状态，一旦拨了第一个数字，电话进入拨号中状态，电话连通后，进入对话状态，话筒放回后，进入空闲状态。

② 由于时间的流逝对象的状态会发生改变。

对于人这个类来说，当某个人处于1岁时，我们认为它处于婴儿期，当他12岁时，我们认为它处于少年期，当他18岁时，我们认为它已经是青年了

状态还描述对象对输入事件的响应。响应可能包括行为调用或状态变化。



■状态的图形表示

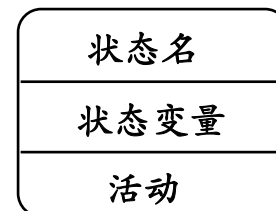
包括三个部分：

状态名

状态变量：有时状态由一些特征变量表示，这时，可以将这些特征变量定义为状态的变量。表达状态的特征值

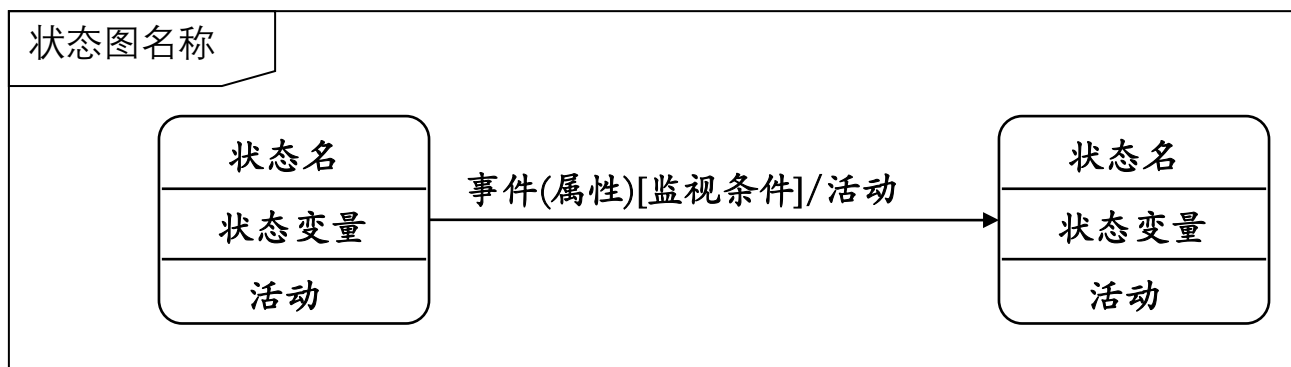
活动：

- 进入该状态时对象的活动
- 退出该状态时对象的活动
- 处于该状态时对象一直进行的活动



■状态图

状态图由对象各个状态和连接这些状态的转换组成

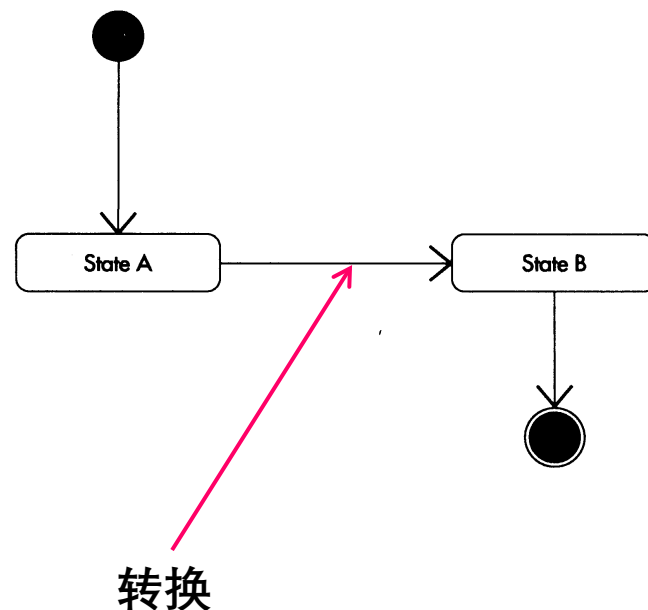


■ 转换（或称迁移）

用来显示从一个状态到另一个状态的处理流。转换使用从一个状态到另一个状态的开放箭头来标记，如下图所示。

转换的五要素：

- 源状态：即受转换影响的状态
- 目标状态：当转换完成后对象的状态
- 触发事件：用来为转换定义一个事件，包括变更、信号、时间三类事件
- 监视条件：布尔表达式，决定是否激活转换
- 动作：转换激活时的操



■ 事件

事件是指在某个时刻对某对象发生的事情，经常与动词过去式对应或与某个条件的初始状态对应。事件瞬时发生，与应用程序的时标有关。

事件有几种类型：

(a) 信号事件：（signal event）

发送和接收信号的事件。

任何类型的事件发生实际上是一个对象向另一个对象传递了一个消息，对象之间通过消息来进行通讯。

从接受消息的对象方看，如果接受到消息能够触发一次状态的变化，那么这个接受到的消息就称为“信号”。通常我们关心信号的接收过程，因为它会对接收对象产生影响。

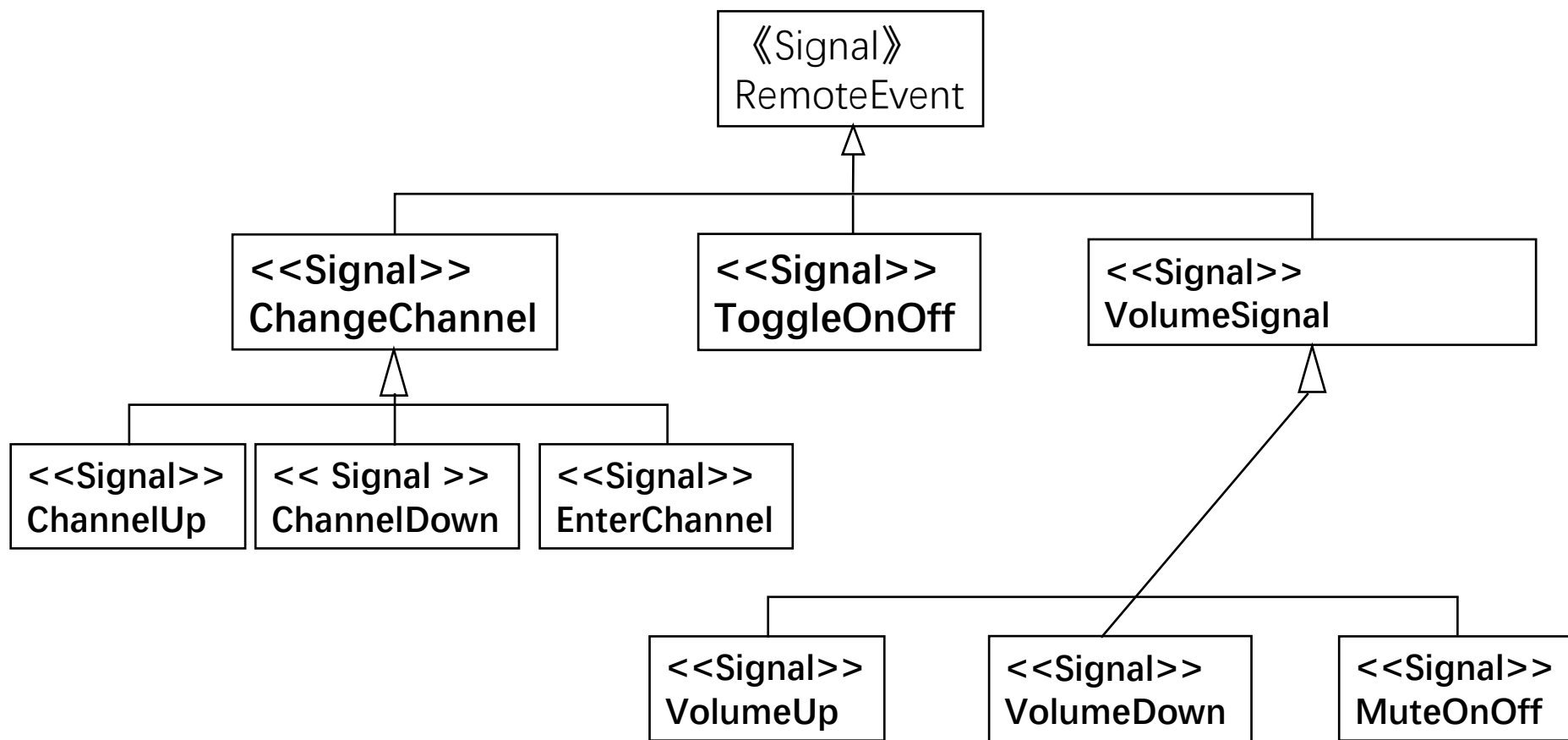
信号可以组织成信号类，也可以有自己的属性和行为。

```
<<signal >>  
MouseButtonPushed  
  
button  
location
```

信号泛化

把信号组织成类，建立信号之间的继承层次

例子：用来操纵电视机的遥控器



(b) 变更事件: (change event)

由满足逻辑条件表达式而引起的事件。不断测试条件，只要条件满足，事件就发生。
变更事件的UML表示法由**when(条件表达式)**来表示。

- when(room temperature < heating set point)
- when(tire pressure < minimum pressure)
- when(battery power < lower limit)

(c) 时间事件 (time event)

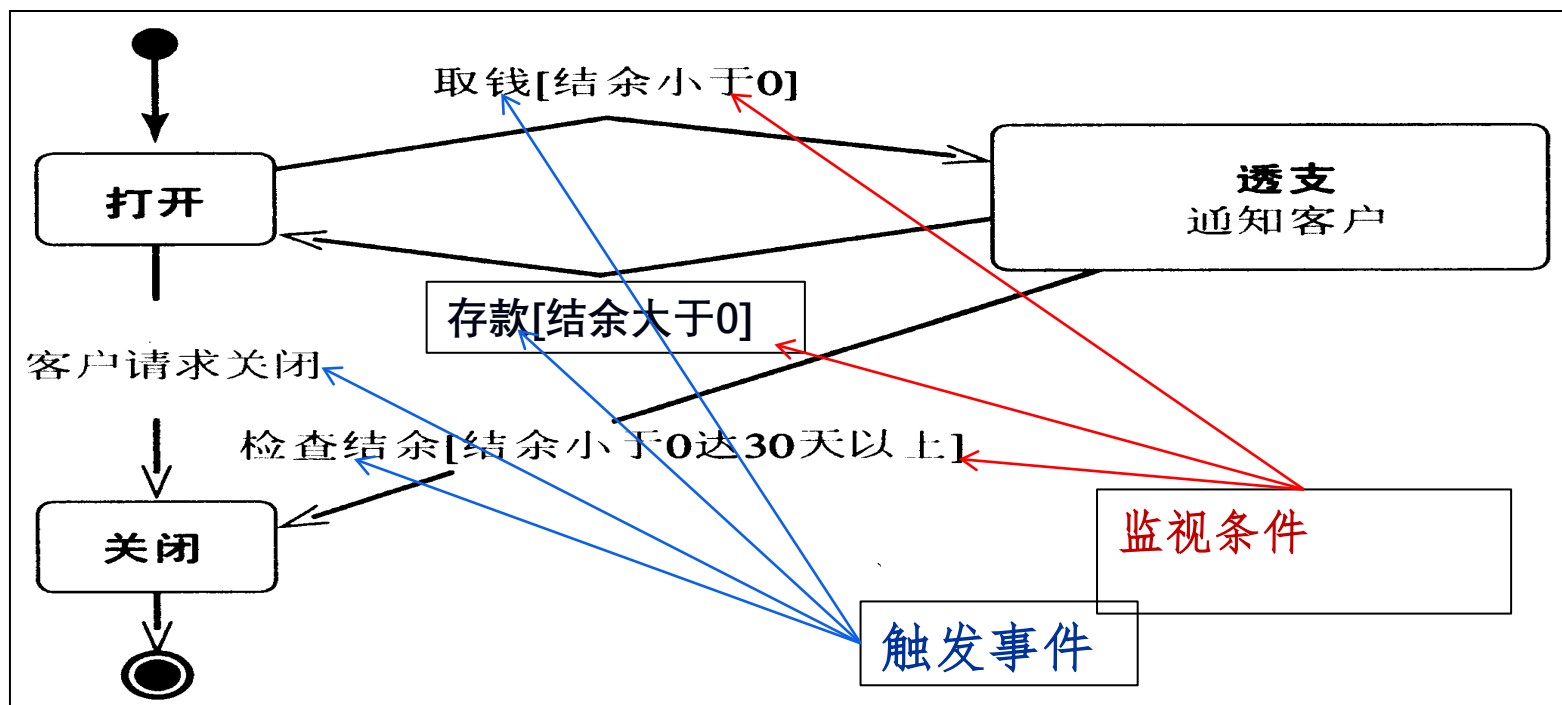
在绝对时间上或在某个时间间隔内发生的事情而引起的事件。

其表示法为：

- 绝对时间的UML表示法采用关键词：**when(时间表达式)**
when(date = January 1, 2000)
- 时间段的UML表示法是采用关键词：**after(时间间隔)**
after(10 seconds)

■ 监视条件 (guard condition)

使状态发生变化的条件，一般为布尔表达式。当事件发生时，监视条件就会检查一次，当条件满足时就触发转换（从一个状态变化到另一个状态）。



■ 活动

活动是为响应事件而执行的行为，活动执行的条件包括：进入(entry)或退出(exit)某状态以及收到状态内的其他事件(do)。

活动也可以表示对象内部控制的操作。如设定属性或生成其他事件，常见的例子有：每次执行了某种特殊事件，程序就会累加内部计算器。

活动的表示法为在事件名后加“/活动名”

一般地，一个状态有三个基本的活动：

(a) do活动：

do活动指持续一段时间的一项活动。do活动只会在某个状态内出现，不能附加到转换上。例如警示灯在打印机卡纸时会闪烁。

表示法为：“do/活动名”

Paper jam
do/flash warning light

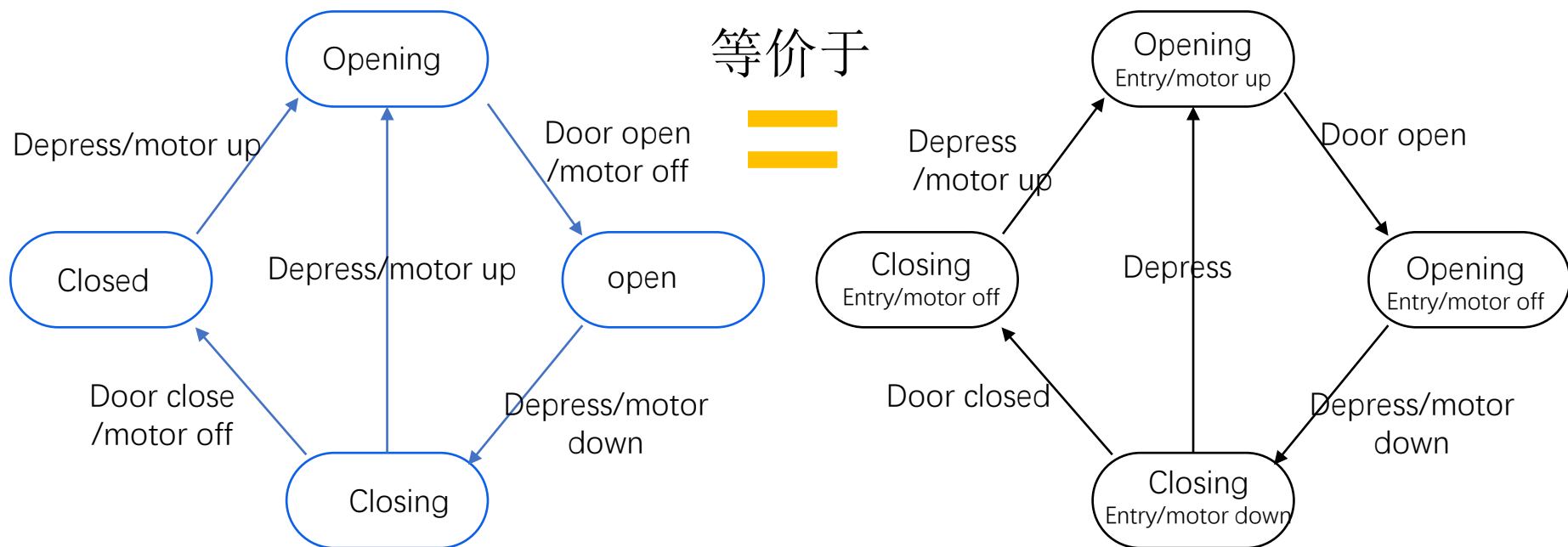
(b) entry活动：

当进入一个状态时，执行某个动作；UML使用“entry/ 活动名”

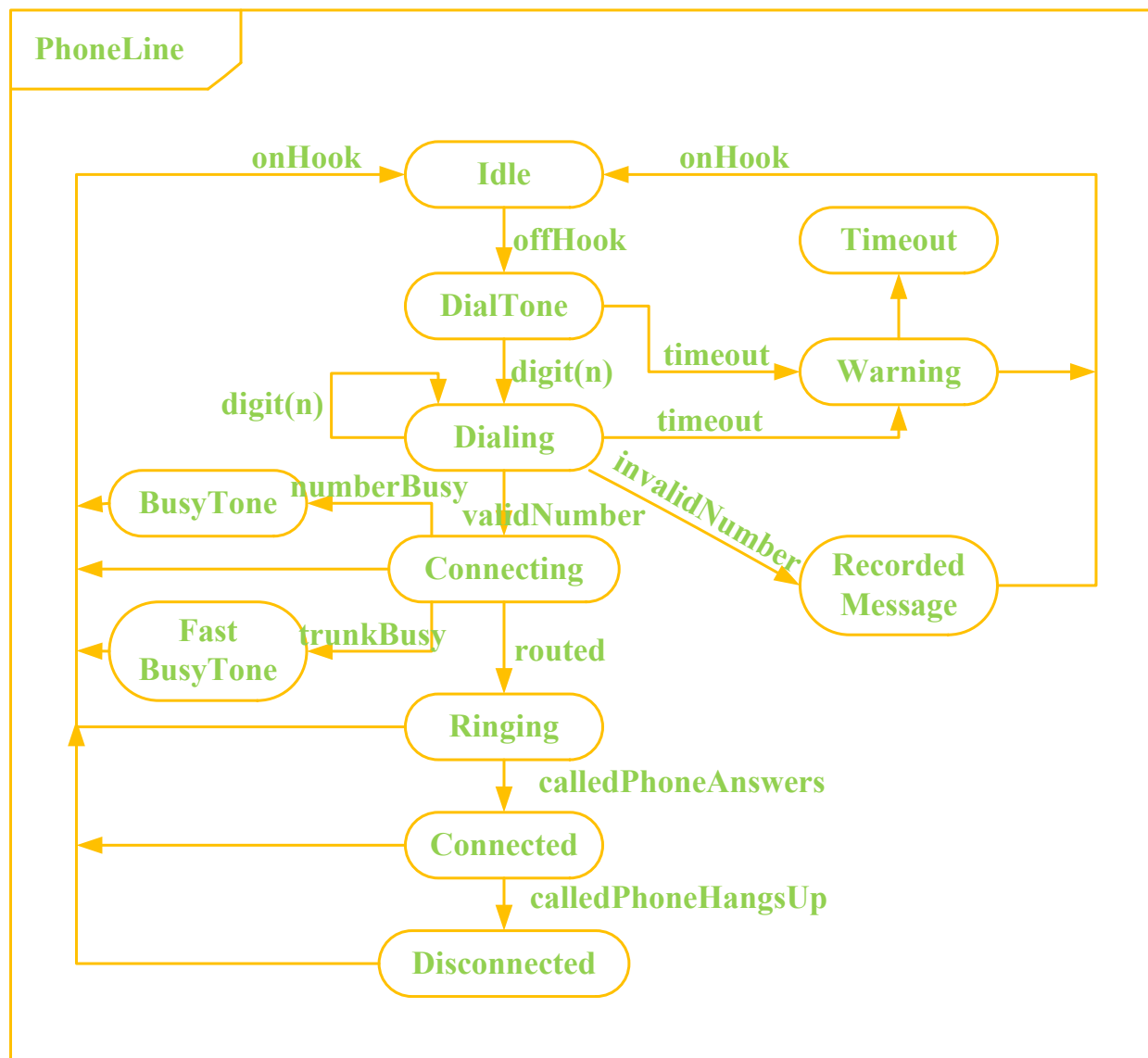
(c) exit活动：

当退出某个状态时，执行什么动作。UML使用“exit/活动名”表示。

车库门的状态图：显示车库开门器的控制过程。用户用按钮生成depress事件来打开和关闭大门，每次事件会反向操作大门的方向，但为了安全起见，在关闭之前，大门必须完全打开。控制流程会产生马达的活动：**motor up**、**motor down**和**motor off**。在活动完成后，马达会产生事件开门(**door open**)和关门(**door closed**)。进入状态**opening**的两种转换都会打开大门。

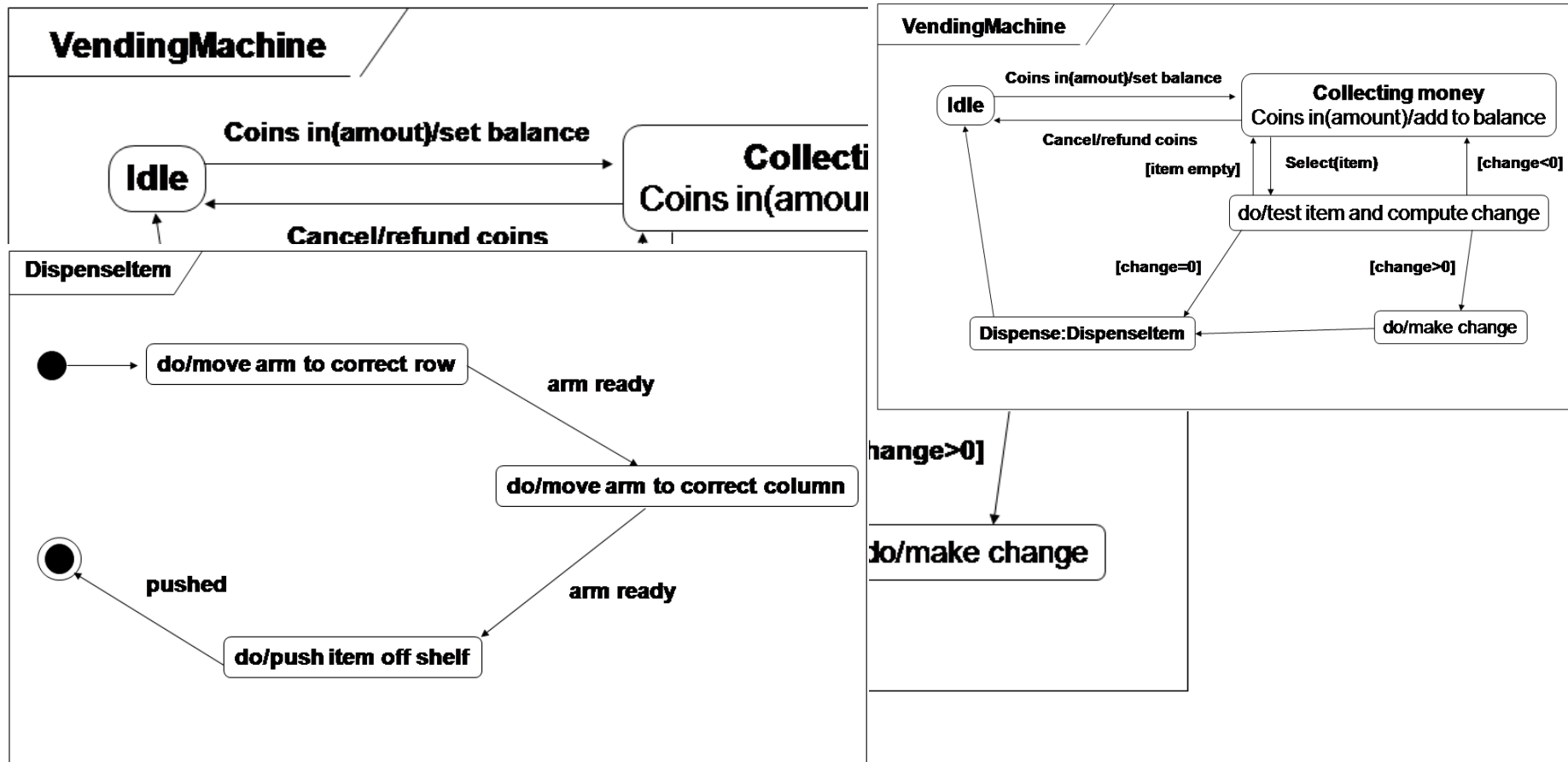


电话的状态图：



■ 嵌套的状态图

当系统中对象的状态、状态之间的迁移太多时，常规的状态图看起来相当复杂，此时，可以利用结构化的方法将状态图分层次地组织起来。高层次的框图简洁地表达整体系统的状态变化，低层次的框图详细的表达部分的状态变化。



■ 嵌套状态（子状态）

可以通过两个例子来看嵌套状态：

(a) 电话机

可以将电话机分为两种状态：空闲状态和激活状态，处于激活状态时，只要压下电话，电话机就变化到空闲状态。激活状态又分为拨号状态、等待状态、连接状态等等，这些都可以认为是激活状态的子状态，它们共享激活状态的行为。

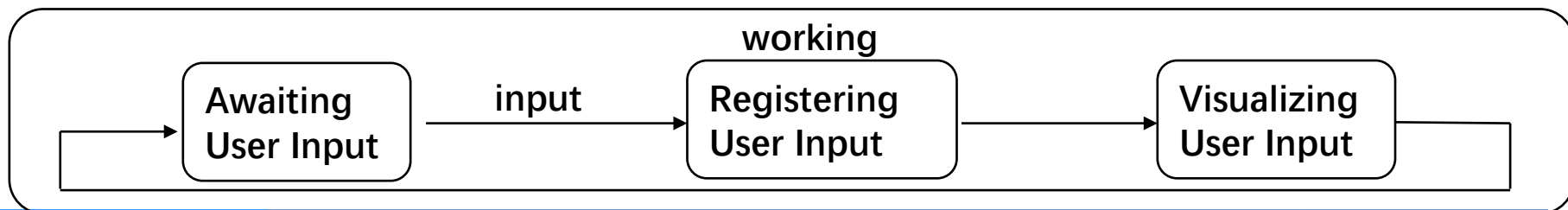
(b) 计算机的图形用户接口（GUI）

当GUI处于Working状态时，它始终在等待用户动作-键盘、鼠标移动、鼠标按钮等待，当某个动作发生时，GUI内部要经历许多变化，如果要对这些内部的变化详细描述出来，还要列出许多状态及状态的变化，那么，这些详细的状态就是Working状态的子状态。

子状态以两种形式出现：**顺序子状态**、**并发子状态**

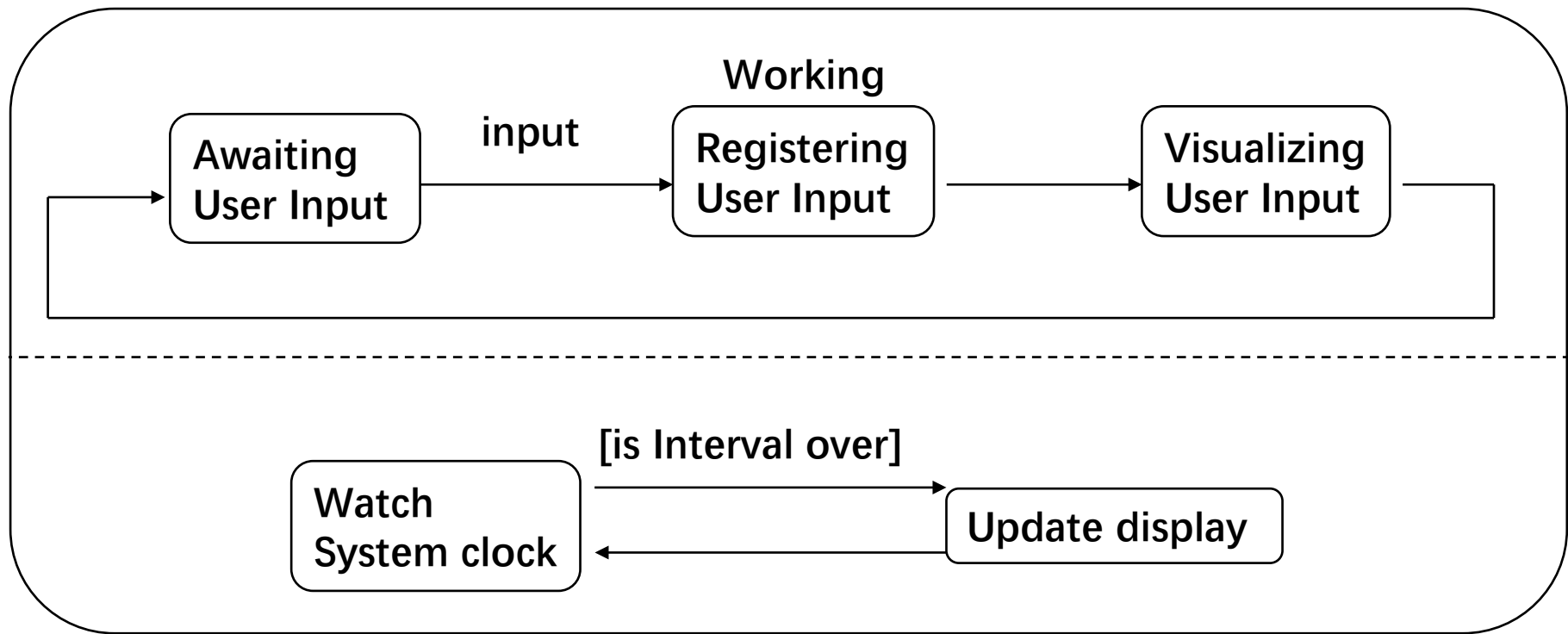
顺序子状态:顺序子状态按照顺序一个接一个出现

如：GUI的Working状态中，对输入事件的响应时表现的内部状态变化为：等待输入—登记输入—显示用户输入。



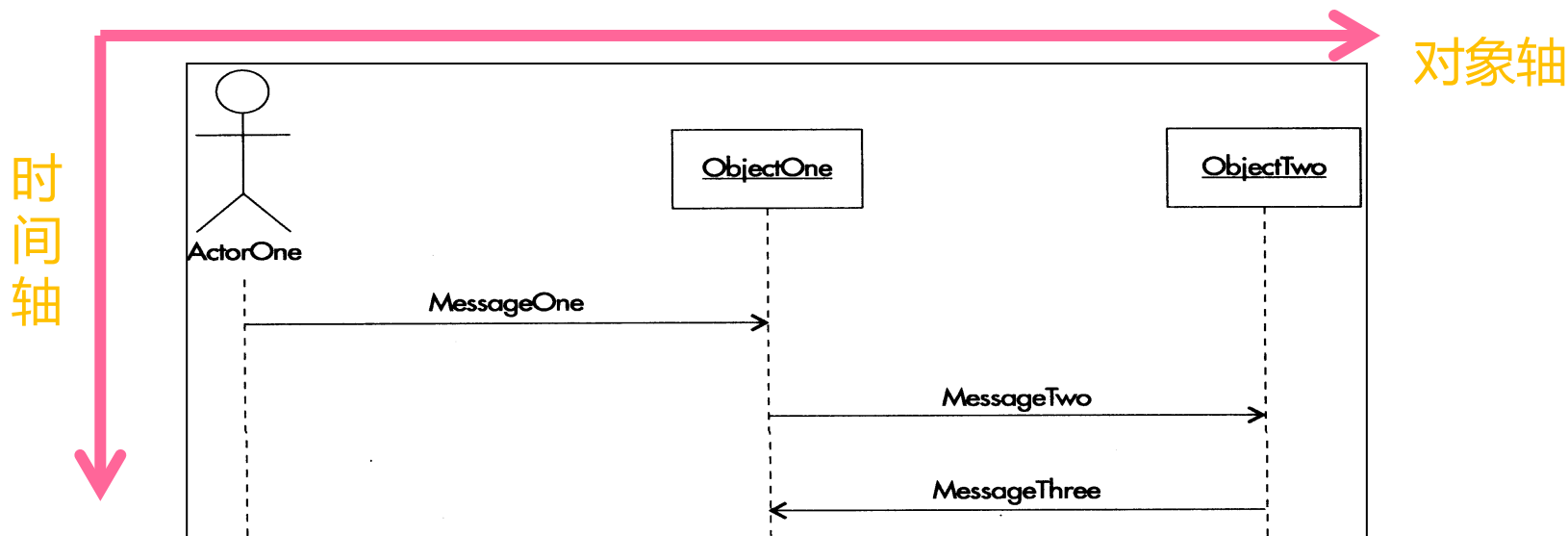
■ 并发子状态

在GUI的Working状态时，GUI并不仅仅是等待用户的输入，它还要监视系统的时钟，一旦达到某个监视条件，它就要定期地更新显示。那么等待用户输入及之后的子状态变化序列与监视系统时钟及之后的状态变化序列是并发的。



◆ 4.2 顺序图 (Sequence Diagram)

- 将对象之间的交互关系表示为一个二维图。
- 纵向是时间轴，时间沿竖线向下延伸。
- 横向轴代表了在协作中各独立对象的类元角色。
- 类元角色用生命线表示。当对象存在时，角色用一条虚线表示，当对象的过程处于激活状态时，生命线是一个双道线。
- 按照时间顺序显示对象之间的交换关系。
- 显示场景中的对象和类以及完成场景中定义的功能时对象间要交换的信息。



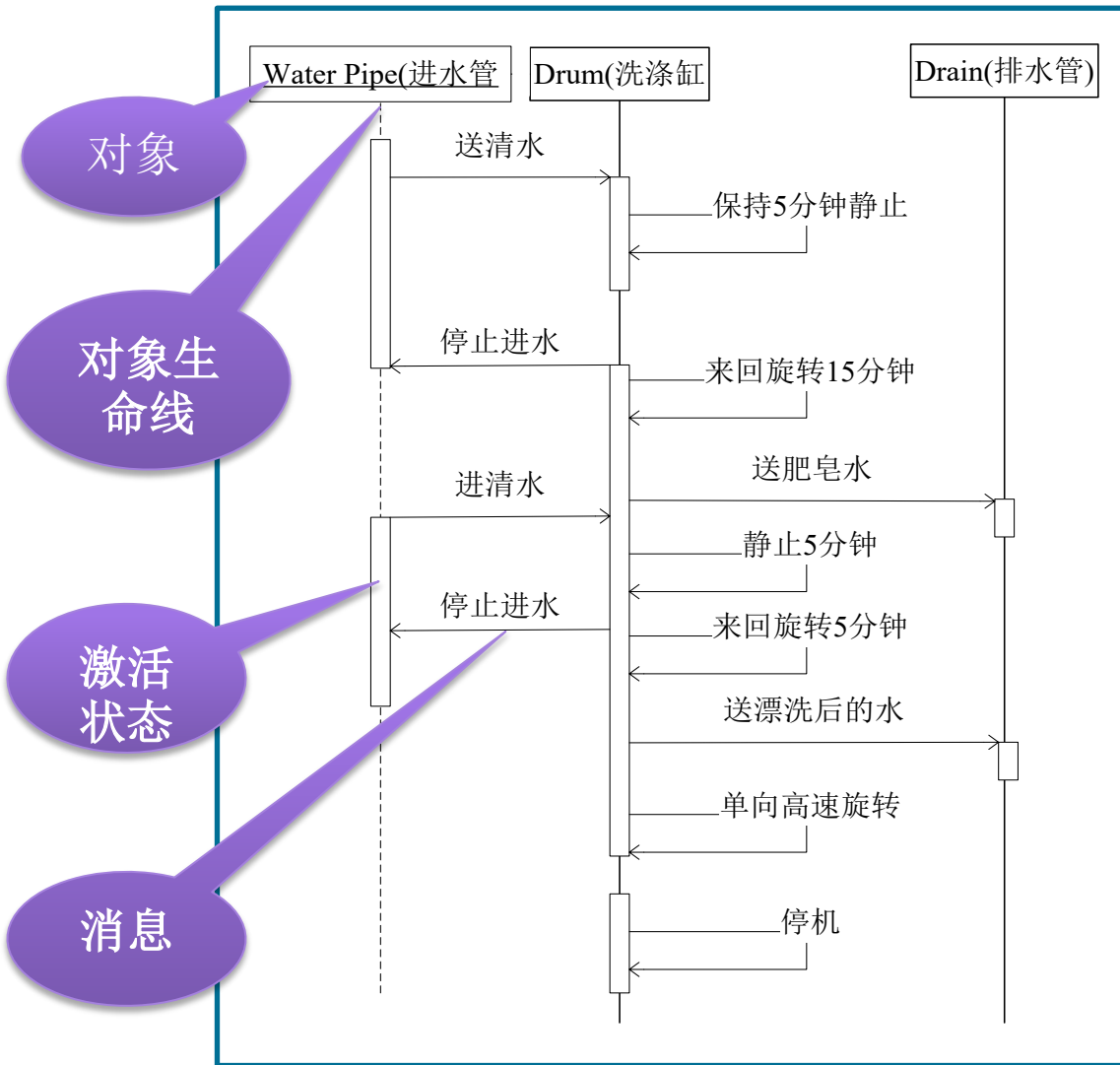
顺序图由对象、消息、时间轴等表示组成；

● 对象

对象从左向右布置在顺序图的顶部，每个对象向下方伸展的虚线叫做生命线，**生命线从上到下表示时间序列**，在生命线上的窄矩形条称为激活。激活表示该对象正在被执行某个操作。激活矩形的长度表示出激活的持续时间。

● 消息

一个对象到另一对象的消息用跨越对象生命线的消息线表示。对象可以发送消息给自己---也就是说消息线从自己生命线出发又回到自己的生命线



用例： 购买饮料

小结： 在客户选择并支付后，自动售货机弹出饮料

参与者： 客户

前置条件： 机器正在等待投入货币

描述： 机器启动，处于等待状态，显示消息“投入硬币”。客户把硬币投入机器中。机器显示投入的总金额，并亮起用这些钱可以购买的物品所对应的按钮。客户按下其中的一个按钮。机器弹出对应的物品，如果物品的价格低于投入的金额，找出零钱。

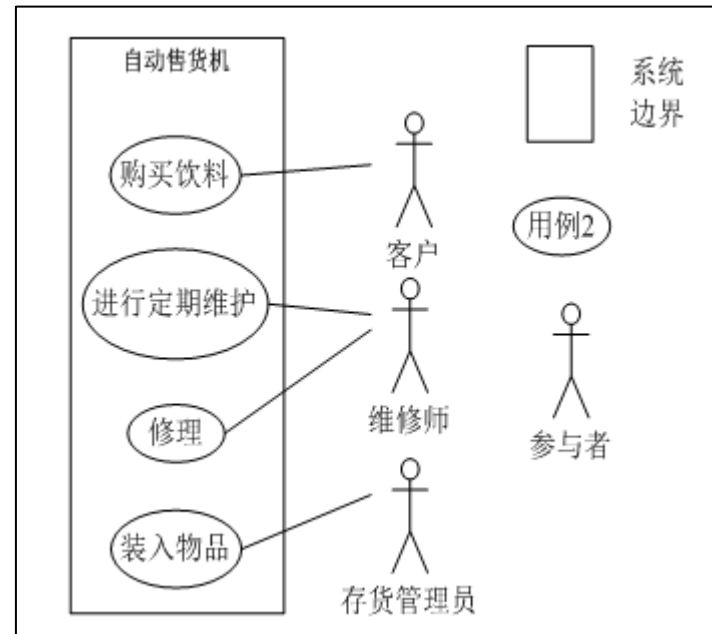
异常：

取消：如果客户在作出选择之前按下了取消按钮，机器会复位到等待状态

脱销：如果客户按下了一个脱销的物品所对应的物品，就会显示“此物品脱销”。机器继续接收硬币或等待选择

金额不足：如果客户按下某物品的按钮，而这个物品的价格高于投入的金额，就会显示消息“金额不足”。机器继续接收硬币或等待选择

后置条件： 机器等待投入硬币



自动售货机用例图

假设在自动售货机上有3个对象来做上述工作：

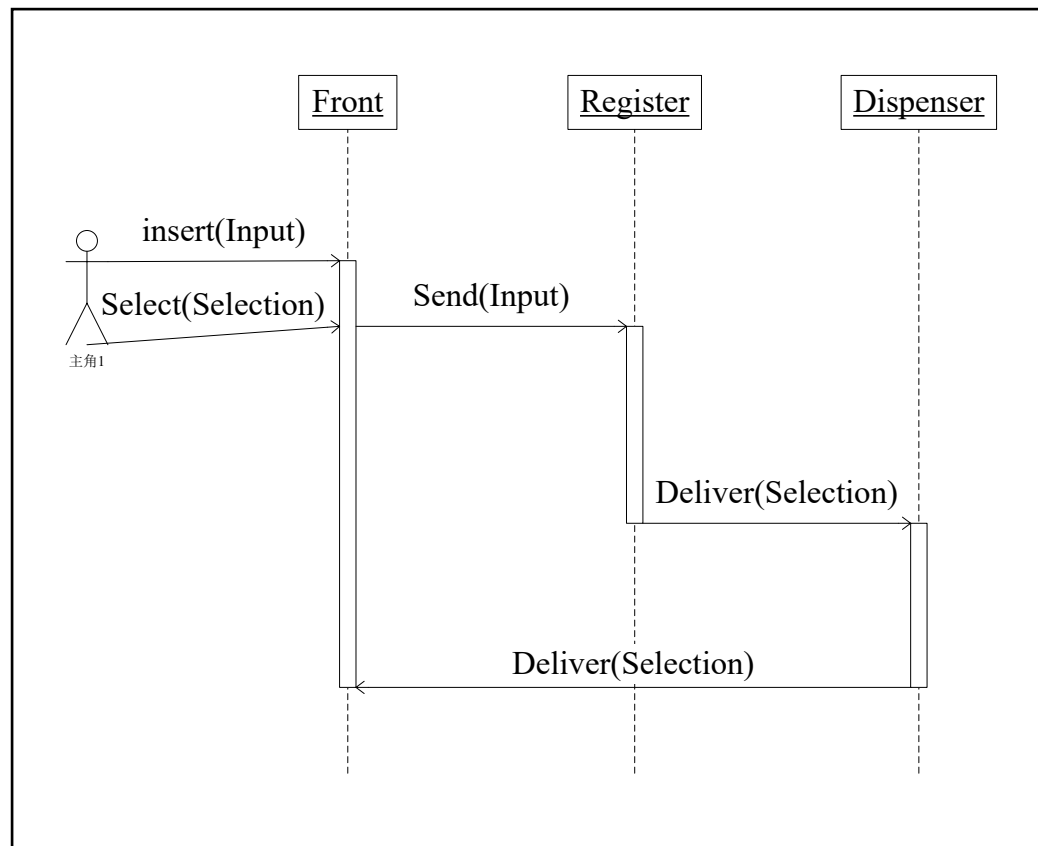
前端 (front) --- 自动售货机与顾客之间的接口

钱币记录仪器 (Register) -- 负责收集顾客投的钱币

饮料分发器 (Dispenser) -- 负责向顾客弹出购买的饮料，受钱币记录仪器控制。

如果只考虑正常情况，对象之间的交互序列可能如下所示：

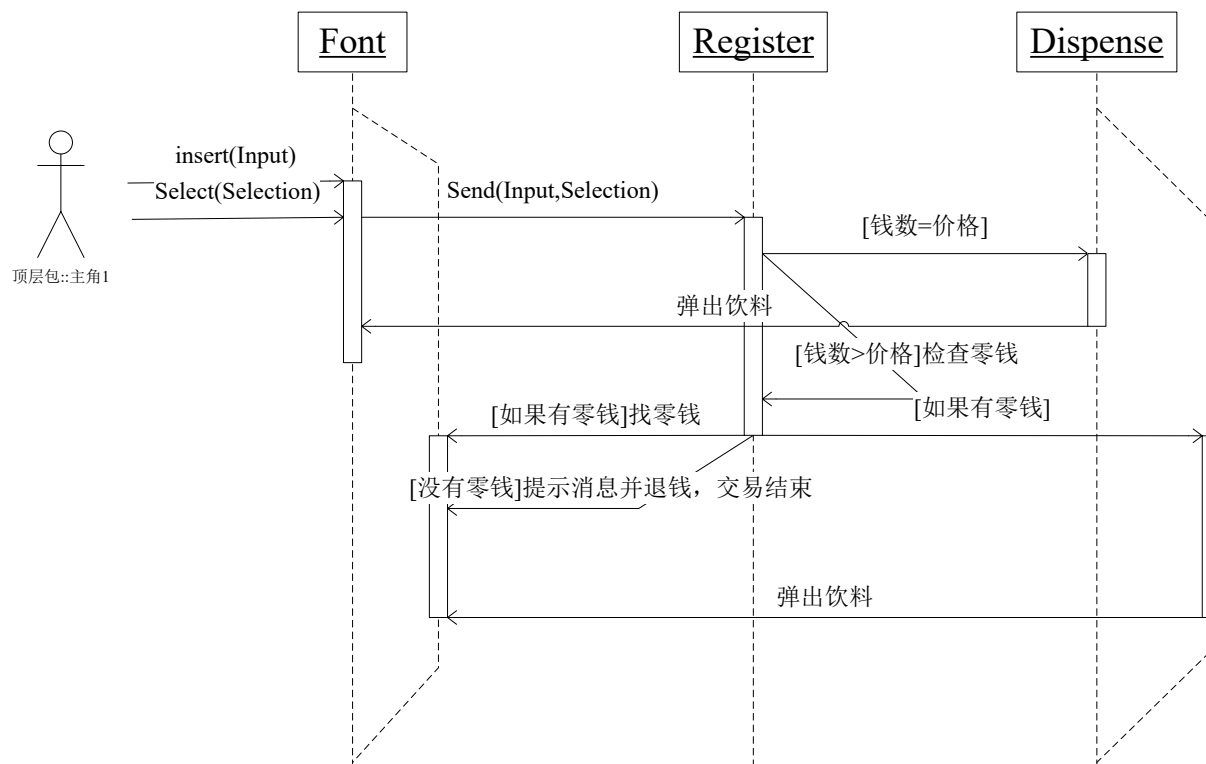
- (1) 顾客向机器前端投入钱币
- (2) 顾客选择所要的饮料
- (3) 钱币被转入到钱币记录仪
- (4) 钱币记录仪器控制饮料分发器向顾客弹出饮料。



若考虑该用例的变体:

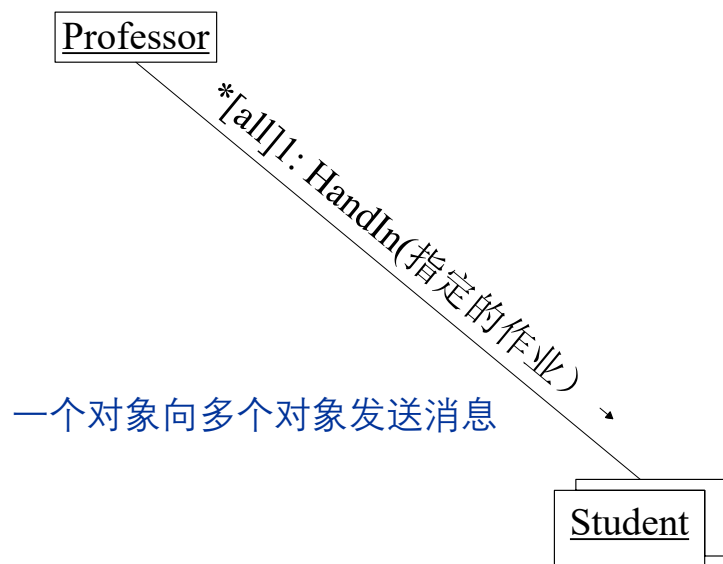
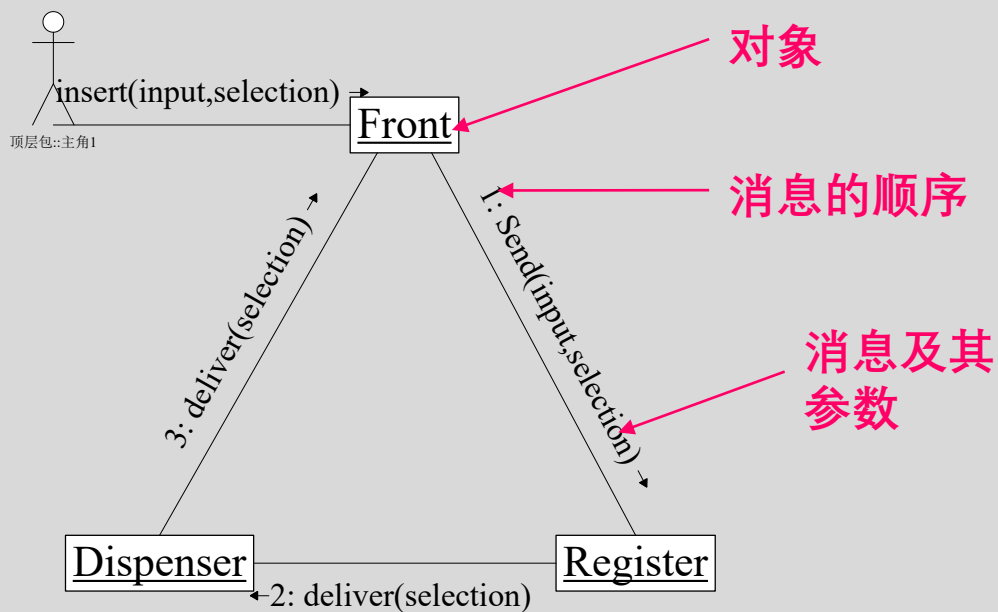
(1) 钱数不正确, 设其场景为:

1. 记录仪检查顾客输入的钱币是否与所要的饮料价格匹配
2. 如果输入数大于价格, 则记录仪计算差值并检查机器中存有的金额
3. 如果机器中有足够的钱找给顾客, 则记录仪将零钱找给顾客
5. 如果机器中没有合适的钱找给顾客, 则记录仪器退钱给顾客, 并显示信息
5. 如果顾客输入钱少于价格, 记录仪什么也不做, 等待顾客继续投币。



◆ 4.3 协作图

协作图(Collaboration Diagram /Communication Diagram, 也叫合作图)是一种交互图(interaction diagram), 强调的是发送和接收消息的对象之间的组织结构, 主要描述对象之间的关联和发送的消息以及发送消息的顺序。



协作图与顺序图的功能一样, 更适合于对象较多的情况。

◆ 4.4 活动图

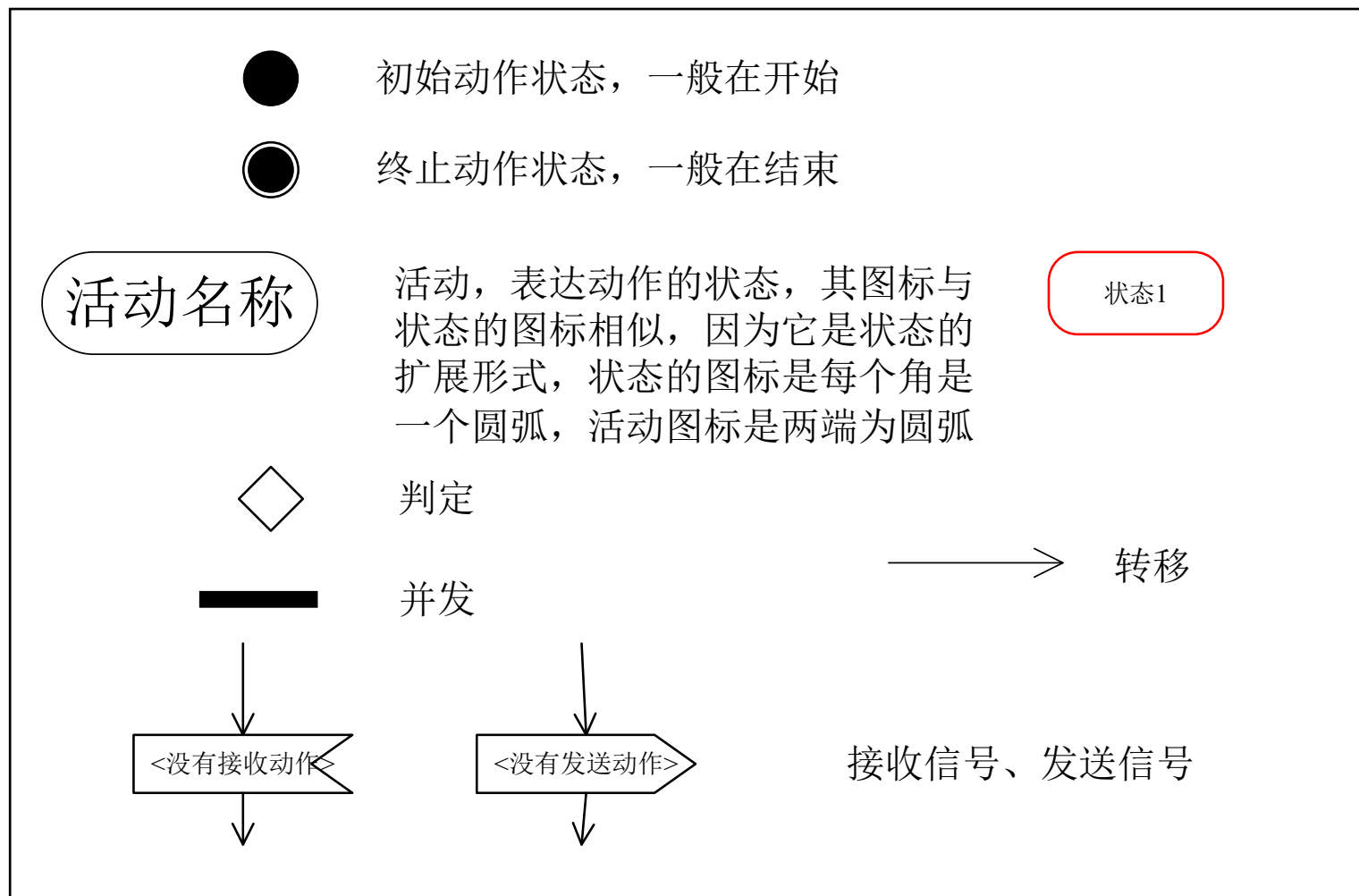
程序流程图表示一个步骤序列、过程、判定点和分支，在结构化设计中主要有：顺序、条件、循环。通常提倡程序设计新手应使用流程图作为可视化描述工具来表达问题并导出问题解决方案，程序流程图是描述算法的工具、是编写程序代码的基础。

UML中的活动图，与程序流程图很相似，它主要显示出工作步骤（将工作步骤程序活动）、判定点和分支。可以用于描述业务过程和类的操作。

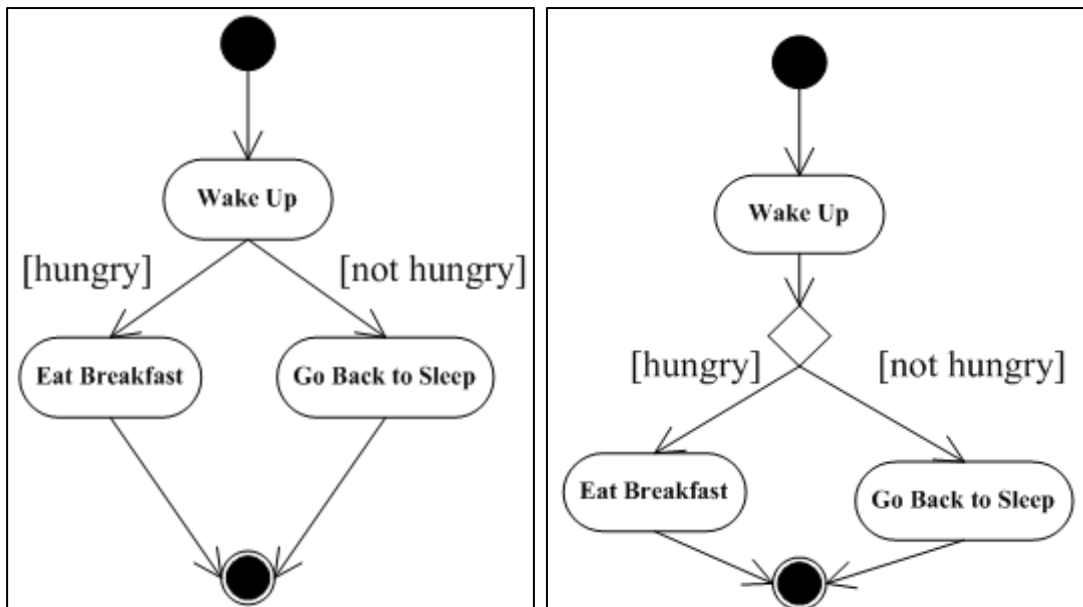
活动图与流程图一样，展示的是控制流，主要专注于操作，顺序图展示的也是控制流，但主要专注于对象。

活动图（activity diagram）显示了组成复杂过程的步骤序列，如算法或工作流。活动图专注于操作而不是对象。活动图在设计算法或工作流的早期相当有用。

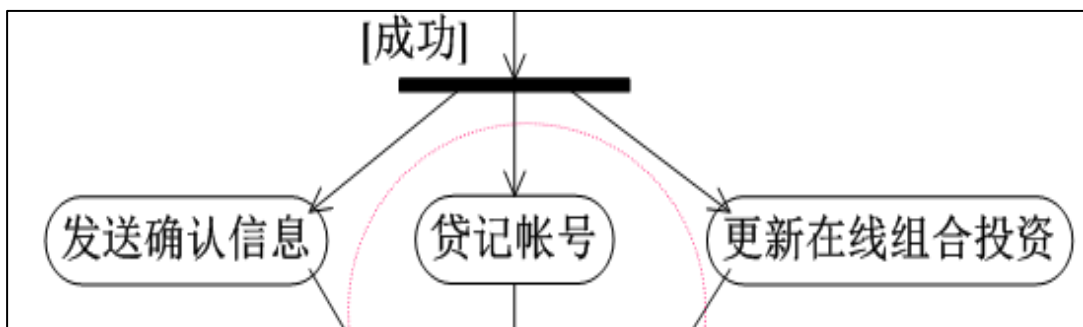
活动图的图例



●判定表示

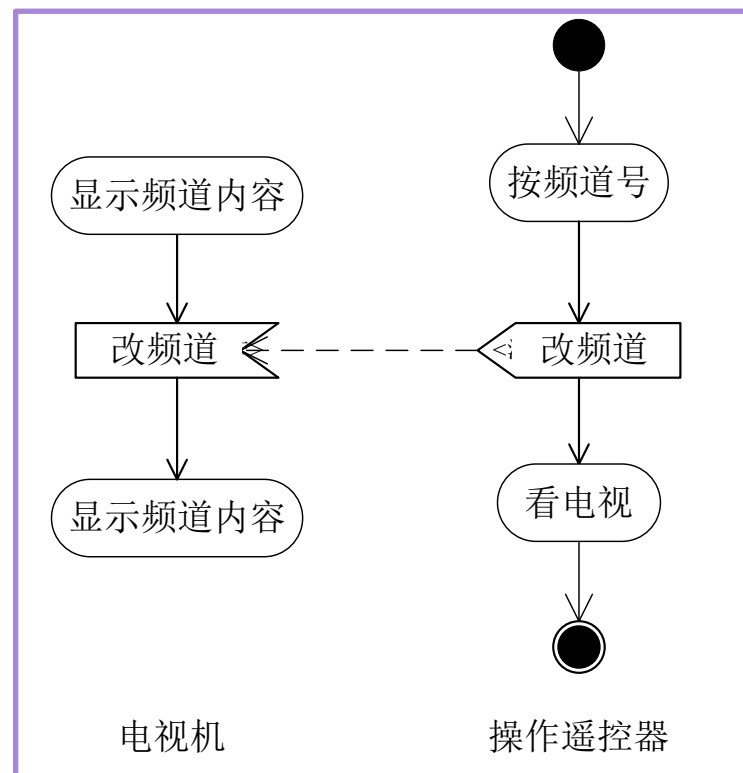


●并发路径表示



●发送信号

活动序列中的活动可以发送信号。当信号被接收时会引起一个活动的发生。信号的图符分发送信号与接收信号两种如下图所示。



● 一个活动图例子

考虑一个咨询公司会见一个新客户的业务过程。活动可能是这样的：

公司业务员打电话给客户，约定会面时间和地点；

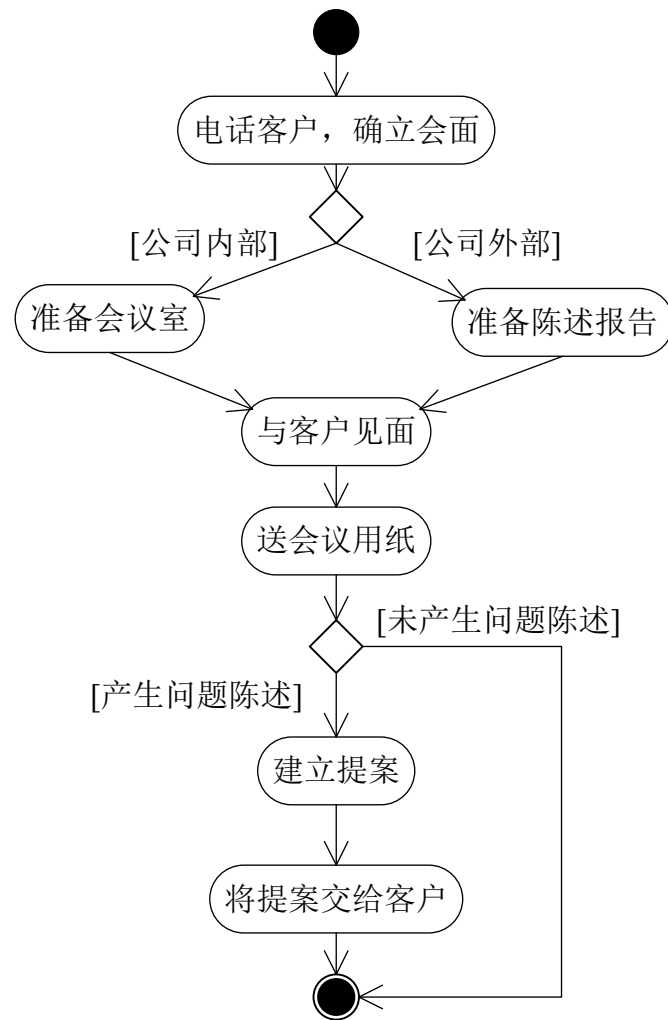
如果约定地点在公司之内，那么公司中的技术人员就要为会面准备一间会议室；

如果约定地点在公司之外，那么咨询顾问就要用笔记本电脑准备一份陈述报告；

咨询顾问和业务员与客户在约定的时间和地点见面；

业务员随后给他们准备好会议用纸；

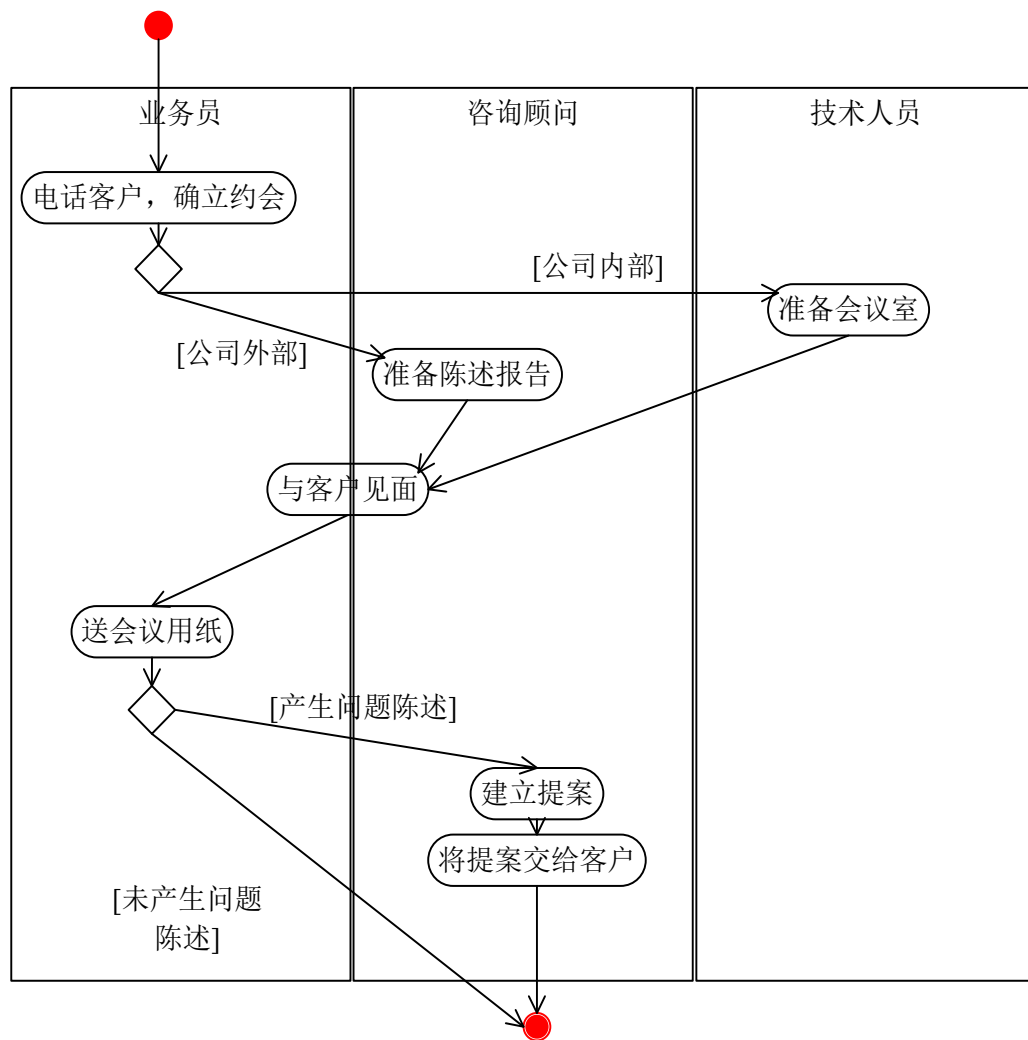
如果会议产生了一个问题陈述，咨询顾问就根据问题陈述建立一个提案，并把该提案发给客户。



◆ 泳道图

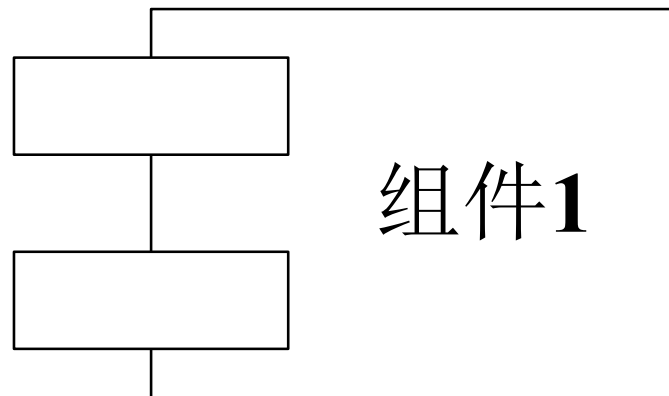
从上面的例子，我们可以看出，尽管在过程的文字描述中我们对各个活动的负责者都给于了描述，但在活动图中得不到体现。

泳道图在活动图的基础上，添加活动的角色信息，使得每个活动的负责者能很明确的表达出来。其方法为将活动涉及的角色从左到右布置，每个角色占一列，称为一个泳道，在活动图中将该角色负责的活动放置在其所占的泳道中。如下页图所示



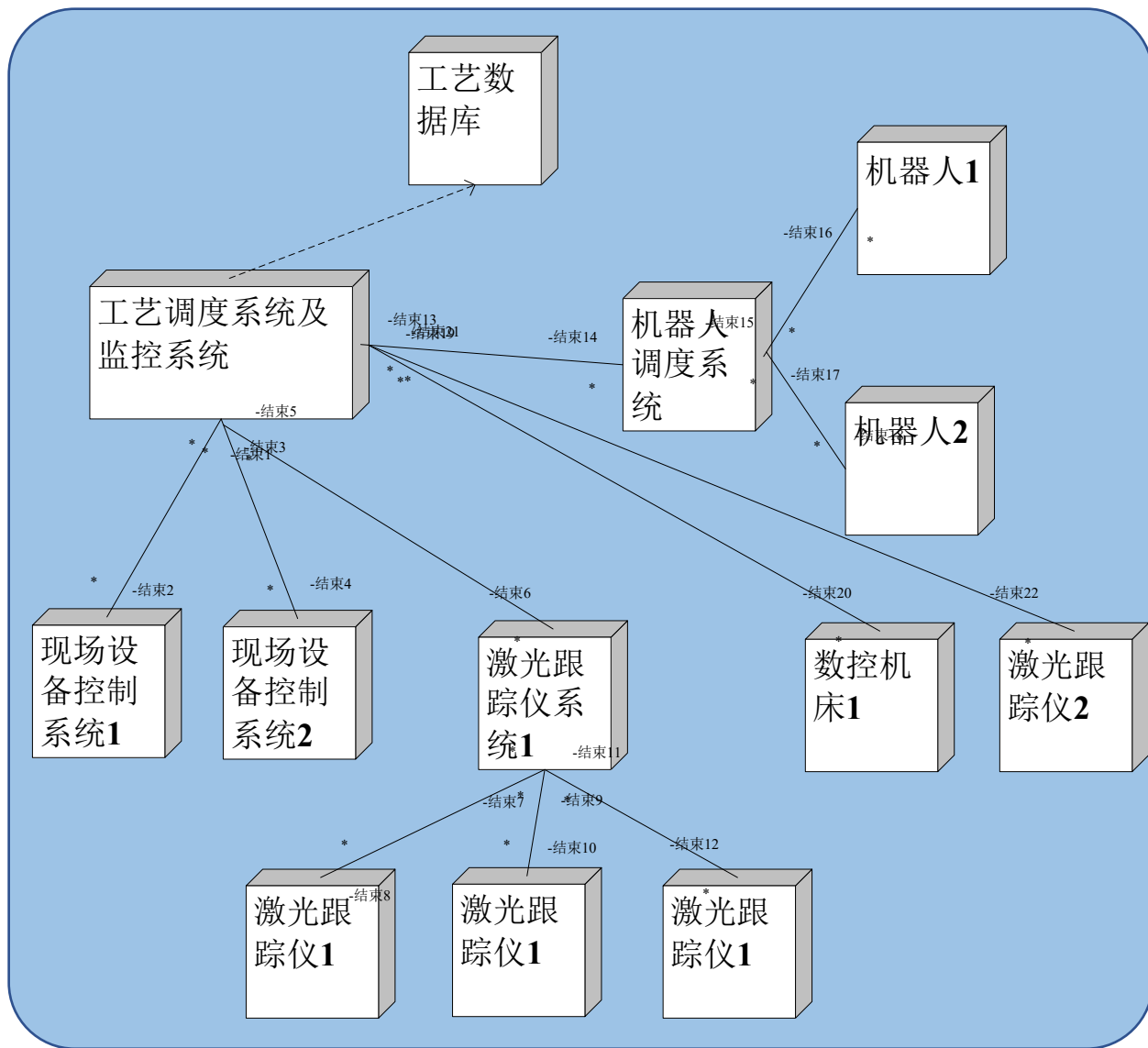
构件图（组件图）

- 组件图与整个计算机系统密切相关。
- 现代软件开发是基于组件的，组件是软件系统的一个物理单元，是驻留在计算机中的关于一组类的实施的程序。
- 如：C++中的动态链接库，C#中的包；
- 将一些类的实现组合成组件的目的是为了重用。
- 右图为UML中组件图的表达。



部署图

UML部署图反映了基于计算机系统的物理体系结构。它可以描述计算机和设备，展示它们之间的连接，以及驻留在每台计算机中的软件。每台计算机用一个立方体来表示，立方体之间的连线表示这些计算机之间的通讯关系。





UML中组织和扩展模型图的一些特征

主要有：

包

注释

构造型

(1) 包 (package)

包表示系统模型中类的分组情况，每个包为一组。实践中，常可以用来表示子系统。

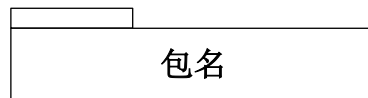
(2) 注释

(3) 构造型

构造型是双尖括号括起来的字符串

它用于扩展一个UML元素。

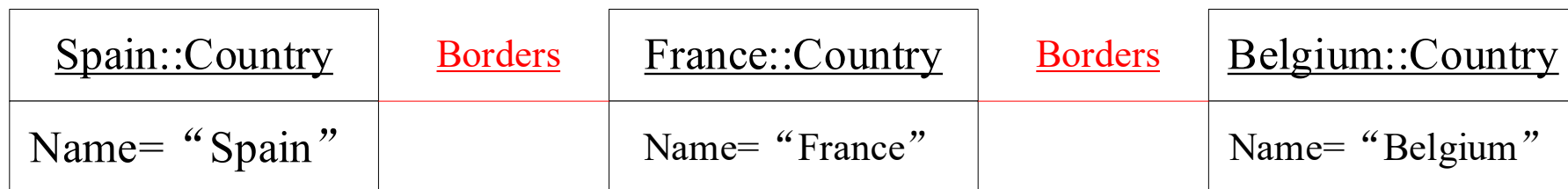
注释的例子
这是一个包的图标，
其包名为“包名”



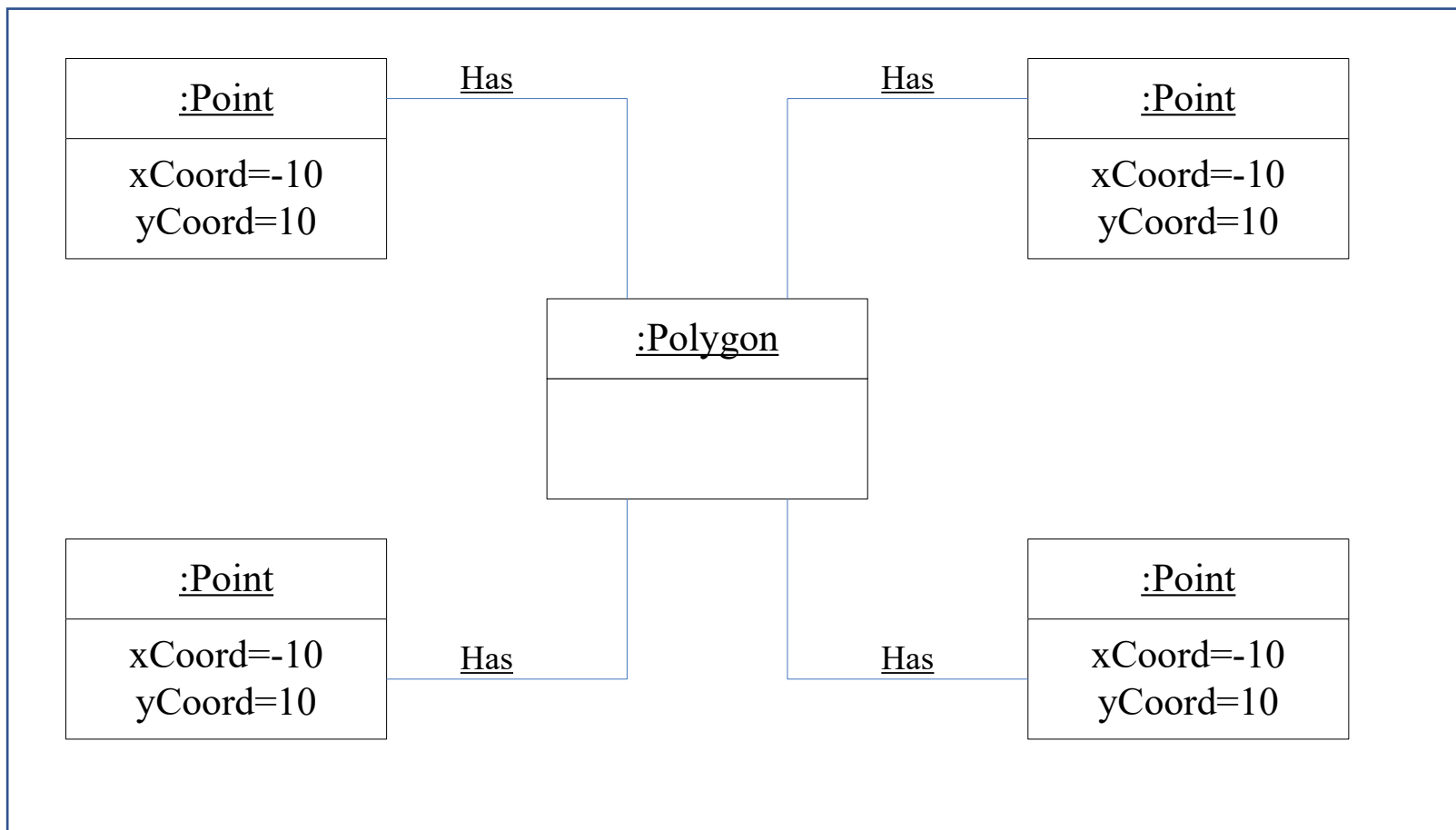
1、学校中有若干学院，每个学院有若干系，每个系有若干个班级和教研室，每个教研室有若干个教师，其中一些教授和副教授每人带若干个研究生。每个班有若干个学生，每个学生选修若干课程，每门课程可由若干学生选修。

抽象出以上模型类图。

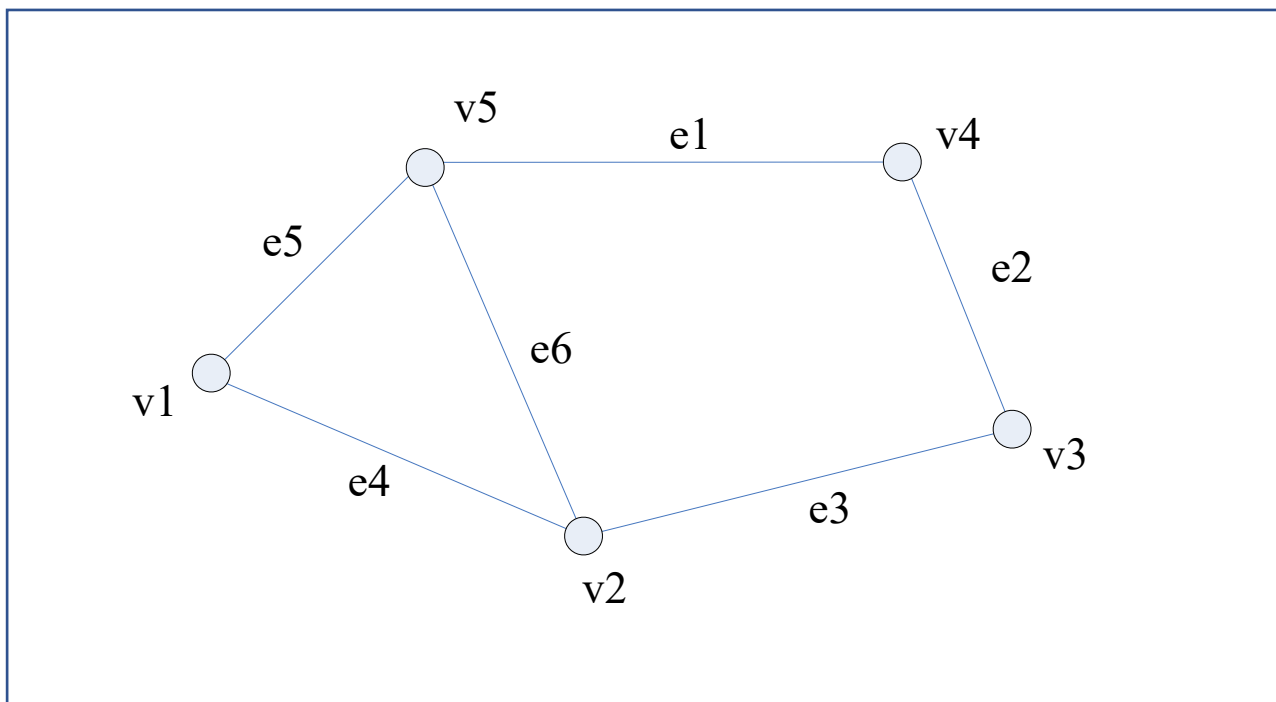
2、根据图1所示的对象图，绘制出类图。



3、根据图2所表示的对象图，绘制出类图。注明关联上的多重性。图要反映构成一个合法多边形最小需要多少个点及点是按顺序排列的事实。



4、绘制一个类模型来描述无向图。无向图由一组顶点和边组成。边连接顶点对。模型应该只捕获图的结构（即连接性），不需要考虑顶点位置和边长等外形问题（图3）。



5、为下面的每一组类绘制一个类图。需要表达类间关联、聚集、组成、泛化等关系、以及多重性、角色名等内容。可以添加类。

A：学校、操场、校长、教室、书本、学生、教师、餐厅、公共厕所、计算机、课桌、椅子、尺子、门、秋千。

B：汽车、引擎、车轮、刹车、刹车灯、车门、电池、消声器、排气尾管

C：文件系统、文件、ASCII文件、二进制文件、目录文件、磁盘、驱动器、磁道、扇区

6、把下列关系分类为：泛化、聚合、关联。

A 一个国家有一个首都

B 用餐者使用筷子

C 文件要么是普通文件，要么是目录文件

D 文件包含记录

E 多边形由有序点组成

F 绘制的对象是文本、几何对象

G 鼠标和键盘是输入设备

H 某学生选择了某教授的课程

7、下面是旋转电机的分类。为了分析方便，电机分为交流电机、直流电机。交流电机可以是同步电机、也可以是感应电机。直流电机包含永磁电机和通用电机两种。用类图显示各种电机之间的关系。

8、绘制一个类模型来表示词典中的单词。包括以下内容：拼写、发音、含义、词性（名词、动词、形容词、副词）、词源、其他拼写、同义词、反义词。

9 伸缩梯上有绳子、滑轮和插销，可以升起、放下和锁住。当插销锁住时，伸缩梯会被固定住，你可以很安全地爬上梯子。为了松开插销，需要用绳子稍微升起伸缩梯，然后可以自由地升降伸缩梯。当插销穿过梯子横档时，会发出劈啪的声响。在反方向升起伸缩梯时，插销会重新啮合，就像插销正在穿过横档一样。

绘制一个伸缩梯的状态图。

10.简单的数字手表上面有一个显示屏和两个设置按钮A、B。此表有两种操作模式：显示时间和设置时间。

在显示时间模式下，手表会显示小时和分钟，小时和分钟有闪烁的冒号分隔。

设置时间模型有两个子模式：设定小时和设定分钟。

按钮A用于选择模式。在每次按下A时，模式会连续前进：显示、设定小时、设定分钟和显示分钟。

在子模式内，只要按下按钮B，就会拨快小时或分钟。在按钮生成另一个事件前必须释放它们。

请绘制一个手表状态图。

11 电话的应答控制过程如下

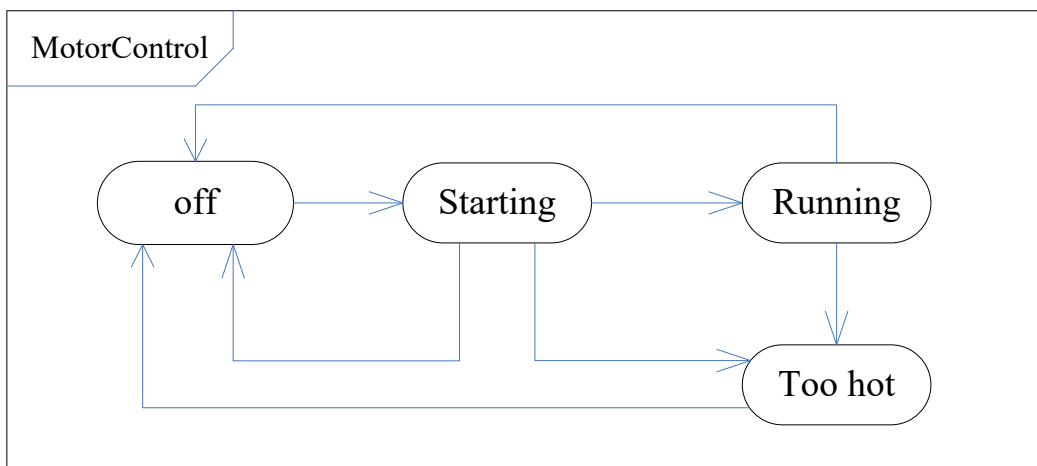
首次响铃时，应答机会检查到达呼叫，然后用预先录制的应答回复呼叫。当应答完成时，机器会记录呼叫端的消息。当呼叫端挂起时，机器也会挂起并关闭。

绘制电话机的状态图。

提示：状态：挂起、回复、记录

12 下图是一个没有完成的状态图，用于描述某种家用电器的马达控制。独立的家电控制回确定马达何时自动启动，并在马达应该运转时连续发出断言on作为马达控制的输入。

- 当on被断言时，马达控制应该启动并且运转马达。接通线圈start和run上的电源，马达启动。传感器starting delay确定马达启动的时间，达到传感器starting delay时，线圈start会关掉，只剩下线圈run上还在加电。当on没有被断言时，两个线圈都会关闭。
- 如果电器马达超负荷运行或无法启动，它们就会由于过热而损坏。为了防止热损害，马达控制经常还会包括过热传感器。如果马达变得太热，马达控制就会关闭两个线圈的电，并忽略任何on断言，直到有重置按钮按下，并且马达冷却下来。
- 在图中加入下列内容。活动：给线圈run加电 (apply power to run winding)和给线圈start加电 (apply power to start winding).事件：马达过热 (motor is overheated),on被断言 (on is asserted)、on 不再被断言 (on is no longer asserted)、马达运转(motor is running)和重置(reset).条件：马达不再过热 (motor is not overheated)。



13.考虑一个真实的书店，例如在购物中心的那种书店。

- a. 列出结帐系统设计中会涉及到的三类参与者。解释每一类参与者的相关性。
- b. 一个用例是购买商品，从客户的角度，列出在类似的抽象层次上的另一个用例。并用一句话来总结每个用例的意图。
- c. 给书店的结算系统绘制一张用例图
- d. 给每个用例编制一个普通的场景。
- e. 给每个用例编制一个异常场景。
- f. 对于d中的每个场景，绘制一张顺序图。

14 绘制一张活动图，描述如何计算餐厅账单。每次上餐都应该收费。对于6人或更多人的一组，要按总量征税，并收取18%作为服务费。对于更少人的组，应该有一个表示消费的空项，客户自愿支付消费。客户递交的任何优惠券和礼物凭证都要从总费用中减去。

15 绘制一张活动图，描述飞行奖励积分。某某航空公司的飞行奖励积分以里程数记。乘客乘坐该航空公司的飞机，航空公司记录其里程，并累计起来，以作为奖励依据。其记录方法为：

- ◆金卡持有者如旅行里程的125%少于1000公里，按1000的里程数，否则按里程的125%算。
- ◆红卡持有者如旅行里程的150%少于1000公里，按1000的里程数，否则按里程的150%算。
- ◆普通乘客若旅行的里程少于750公里，按750公里计算里程数

2026

THANK YOU

浙江大学机械工程学院航空制造工程研究所
王青 李江雄